Installing OpenBSD

Once you boot from your chosen media, you should see something like this:

```
> OpenBSD/amd64 BOOT 3.18
boot>
```

If you need to interrupt the boot process for any reason, you can do so at this point. We'll discuss how to interrupt the boot process in Chapter 5, and reasons for doing so throughout the book.

If you wait five seconds, OpenBSD should boot. The kernel will then introduce itself and begin identifying your hardware.

```
booting ❶cd0a:/5.3/amd64/bsd.rd: 2986868+913996+2861496+0+504624
[89+318288+205653]=0xb6f578
entry point at 0x1001e0 [7205c766, 34000004, 24448b12, 1608a304]
Copyright (c) 1982, 1986, 1989, 1991, 1993
      The Regents of the University of California.  All rights reserved.
Copyright (c) 1995-2012 OpenBSD. All rights reserved.  http://www.OpenBSD.org

❷ OpenBSD 5.3 (RAMDISK_CD) #23: Sun Feb 12 09:45:07 MST 2012
    deraadt@amd64.openbsd.org:/usr/src/sys/arch/amd64/compile/RAMDISK_CD
real mem = 1072627712 (1022MB)
avail mem = 1032290304 (984MB)
...
```

In this output, you can tell at ❶ from which device the system is booting—CD drive 0 in this case. Next, you see the copyright information, followed by the directory in which your kernel was compiled at ❷. You can see that this is an OpenBSD snapshot kernel, compiled by user `deraadt` on host *amd64.openbsd.org*.

At this point, OpenBSD should probe your hardware and display the results as it attaches device drivers.

Running the Installation Program

Once the boot messages pass, you should see the following text:

```
Welcome to the OpenBSD/amd64 5.3 installation program.
(I)nstall, (U)pgrade or (S)hell? i
```

As you can see, there are three options: Install, Upgrade, and Shell. The OpenBSD installer is a shell script that calls programs to download files, format disks, and otherwise prepare your system. It might not be pretty, but it is extremely fast and, in educated hands, extremely powerful.

The Shell option will drop you into an OpenBSD command line, where you have access to the commands on the installation disk. These minimal commands might suffice to repair a damaged system. We'll examine the Upgrade option in Chapter 20.

Enter **i** to choose Install. You should see a welcome message and a few basic instructions:

```
At any prompt except password prompts you can escape to a shell by
typing '!'. Default answers are shown in []'s and are selected by
pressing RETURN.  You can exit this program at any time by pressing
Control-C, but this can leave your system in an inconsistent state.

❶ Terminal type? [vt220]
❷ System hostname? (short form, e.g. 'foo') caddis
```

The installer shows default answers in square brackets. To use the default, just press ENTER.

If your system has a standard keyboard and monitor, OpenBSD will use it as the standard VT220 terminal, as shown at ❶. If you have an unusual terminal connected to your system, you're probably an old geezer who knows exactly what terminal type it is. If you're a young kid using some ancient, unidentified, dust-covered terminal found in a disused laboratory at the back of an abandoned fireworks factory because you thought it would be nifty, stop now and get a standard monitor and keyboard. While OpenBSD probably supports that antediluvian console, this is not the time to try it.

Next, the installer should prompt you for the system's short hostname at ❷, which will be a single word to identify your system. This particular computer is named `caddis`; you can name yours whatever you like.

Now to configure the network:

```
❶ Available network interfaces are: em0 em1 vlan0.
❷ Which one do you wish to configure? (or 'done') [em0]
❸ IPv4 address for em0? (or 'dhcp' or 'none') [dhcp] 192.0.2.85
❹ Netmask? [255.255.255.0] 255.255.255.128
❺ IPv6 address for em0? (or 'rtsol' or 'none') [none]
  Available network interfaces are: em0 em1 vlan0.
❻ Which one do you wish to configure? (or 'done') [done]
❼ Default IPv4 route? (IPv4 address, 'dhcp' or 'none') 192.0.2.1
  add net default: gateway 192.0.2.1
❽ DNS domain name? (e.g. 'bar.com') [my.domain] blackhelicopters.org
❾ DNS nameservers? (IP address list or 'none') [none] 192.0.2.2 192.0.2.10
```

At ❶, the installer lists the network interfaces it recognizes on your machine. It has found three: `em0`, `em1`, and `vlan0`. The first two, `em0` and `em1`, are network cards. I chose `em0` at ❷, the installer's default, by pressing ENTER. Avoid configuring a virtual local area network (VLAN) during installation if possible, especially on your first installation. If you need a VLAN to connect to the Internet, see Chapter 12.

When asked at ❸ if you want to give a static IP address, you can choose to use DHCP by pressing ENTER. I chose to enter a static address because I'll be using this machine as a server. (If you don't need a static address, you can just let DHCP automatically assign you an IP address.)

When you use a static address, you must also enter a netmask at ❹ and (if desired) an IPv6 address at ❺. Now, having configured one network card, OpenBSD asks at ❻ if you've finished configuring the network. If you wanted the installer to walk you through configuring the second network card, you would enter `em1` instead of accepting the default of `done`.

If you assign a static IP address, you must also configure a static route if you want to access the Internet, as shown at ❼. Similarly, you need to tell your host its domain name at ❽ and the IP address of at least one name server at ❾.

At this point, you should be on your local network. If you can't access the network, you probably entered something incorrectly. If nothing else, you can use an exclamation point (`!`) to interrupt the installation and get a shell prompt. (Chapter 12 discusses OpenBSD's network configuration in greater depth.)

Multiple Network Cards

Our example server has multiple network interfaces. I chose to configure interface `em0` because that machine was in front of me, and if I chose the wrong network card, I could move the cable. But what if you don't have physical access to your machine? If you had two different network cards (say, an Intel and a 3Com), you would have a better idea which card is which, but having two identical cards leaves you guessing which card has a cable plugged into which network.

Luckily, the OpenBSD installer lets you escape to a command prompt to do a little investigating. How is this useful here? Network interfaces that are plugged in will tell you what kind of connection they have, and disconnected or otherwise failed interfaces will report that they have no media. Here's how you can interrupt the installer to identify the live interface:

```
Available network interfaces are: em0 em1 vlan0.
❶ Which one do you wish to configure? (or 'done') [em0] !
Type 'exit' to return to install.
❷ # ifconfig
lo0: flags=8008<LOOPBACK,MULTICAST> mtu 33152
em0: flags=8802<BROADCAST,SIMPLEX,MULTICAST> mtu 1500
        lladdr 00:0c:29:aa:09:21
❸       media: Ethernet autoselect (1000baseT full-duplex,master)
        status: unknown
em1: flags=8802<BROADCAST,SIMPLEX,MULTICAST> mtu 1500
        lladdr 00:0c:29:aa:09:2b
❹       media: Ethernet autoselect (none)
        status: unknown
vlan0: flags=0<> mtu 1500
        lladdr 00:00:00:00:00:00
```

Rather than choosing an interface, escape to a command prompt at ❶ by entering an exclamation point (`!`). Then ask OpenBSD at ❷ to tell you about its network interfaces by running `ifconfig`. You can see interfaces `em0` and `em1` in the output. While `em0` reports at ❸ that it's running 1000baseT

at full-duplex, at ❹ you can see that em1 has a media type of none. Interface em0 is plugged in, so that's the interface I want to configure. Enter **exit** to return to the installer, and proceed to configure card em0.

Setting Up Services and the First User

The installer should now ask you to configure some basic system parameters:

```
❶ Password for root account? (will not echo)
  Password for root account? (again)
❷ Start sshd(8) by default? [yes]
❸ Start ntpd(8) by default? [no] yes
  NTP server? (hostname or 'default') [default]
❹ Do you expect to run the X Window System? [yes]
❺ Do you want the X Window System to be started by xdm(1)? [no]
❻ Change the default console to com0? [no]
```

At ❶, enter your root password twice. If the passwords don't match, the installer will make you do it over until they do.

You can enable the Secure Shell (SSH) daemon at ❷ so that you can remotely connect to this machine immediately after installation. If you enable SSH but do not create a user later in the installation, you can SSH to the machine as root. This is a Very Bad Idea when using password authentication and will let intruders more easily compromise your server. If you enable sshd here, be absolutely certain to create a user during the installation process! If you don't, at least disable SSH logins by the root account immediately after installing OpenBSD, as discussed in Chapter 4.

Correct time is important on a network. I usually enable the Network Time Protocol (NTP) daemon ntpd(8) during the installation process, as shown at ❸. OpenBSD chooses a set of publicly accessible time servers by default, but you can specify a local time server if you have one available.

Now tell the installer at ❹ if you intend to run X Windows. X requires that software be permitted fairly broad access into the kernel. If the installer detects a graphic console, it defaults to permitting X. If you don't need a graphic console, disable X access.

If you're running X, you might also want the X display manager xdm(1). At ❺, tell the installer if you want xdm. By default, OpenBSD doesn't start xdm when it boots; you're generally better off installing OpenBSD on your system than configuring X, so I've accepted the default of no here.

If you want this system to use a serial port as the console, you can set that during the installation at ❻. I discuss serial consoles in Chapter 5.

NOTE *For the basic system parameters, I've used the default for all but one. Enabling time service certainly isn't mandatory—I could easily enable ntpd after installation instead. I could have also told the installer to disable X, but I can change that after installation as well.*

Now to set up your first user.

```
Setup a user? (enter a lower-case loginname, or 'no') [no] mwlucas
Full user name for mwlucas? [mwlucas] Michael W Lucas
Password for mwlucas account? (will not echo)
Password for mwlucas account? (again)
Since you set up a user, disable sshd(8) logins to root? [yes]
```

My usual user account name is `mwlucas`. Here, I enter that username, along with a real name entry. The installer creates this account and gives it permission to use the root password (see Chapter 6). You should be prompted twice for the user's password.

NOTE *You're offered a chance to disable root logins over SSH. Use this default. The root account should never be permitted to log in via SSH, unless using public key authentication, and even then, those logins should be restricted. For the reasons to avoid root logins over SSH, do an Internet search for "Hail Mary Cloud."*

Setting the Time Zone

Set your time zone during installation. If you have Internet access when you install OpenBSD, the installer should try to determine your time zone. OpenBSD assumes that the BIOS clock is set in Coordinated Universal Time (UTC). If the BIOS clock is set in some other time zone, you'll need to correct the system time after installation.

I'm in Detroit, Michigan. If you're familiar with US geography, you might think that I need US Eastern Time, but my state has its own time zone.

```
❶ What timezone are you in? ('?' for list) [US/Eastern] ?
  Africa/      Chile/       GB-Eire      Israel       NZ-CHAT      UCT
  America/     Cuba         GMT          Jamaica      Navajo       US/
  Antarctica/  EET          GMT+0        Japan        PRC          UTC
  Arctic/      EST          GMT-0        Kwajalein    PST8PDT      Universal
  Asia/        EST5EDT      GMT0         Libya        Pacific/     W-SU
  Atlantic/    Egypt        Greenwich    MET          Poland       WET
  Australia/   Eire         HST          MST          Portugal     Zulu
  Brazil/      Etc/         Hongkong     MST7MDT      ROC          posix/
  CET          Europe/      Iceland      Mexico/      ROK          posixrules
  CST6CDT      Factory      Indian/      Mideast/     Singapore    right/
  Canada/      GB           Iran         NZ           Turkey
❷ What timezone are you in? ('?' for list) [US/Eastern] US
❸ What sub-timezone of 'US' are you in? ('?' for list) ?
  Alaska          Central         Hawaii          Mountain        Samoa
  Aleutian        East-Indiana    Indiana-Starke  Pacific
  Arizona         Eastern         Michigan        Pacific-New
❹ What timezone are you in? ('?' for list) [US/Eastern] US/Michigan
```

I don't recall my exact time zone, but I know it isn't plain old US Eastern Time. I enter a question mark (**?**) at ❶ to see the available options. I don't recognize any of the time zones listed at ❷ as correct for my city, but I know I'm in a US time zone, so I enter **US**. I don't know what my choices of sub-time zones are, so I enter a question mark (**?**) at ❸ to see the US time zones. And there's Michigan! At ❹, I enter the full time zone name.[2]

Setting Up the Disk

As noted earlier, in a dedicated installation, the installer erases all data on the drive. Unlike most other operating system installers, the OpenBSD installer doesn't warn you about this; it assumes that you understand the implications of repartitioning your hard drive.

For this first installation, we'll use OpenBSD's default partitioning scheme. (We'll discuss custom partitioning later in this chapter.) Our demo server has a single disk. We'll first create an MBR partition on this disk and then add OpenBSD partitions.

```
Available disks are: sd0.
Which one is the root disk? (or 'done') [sd0]
Use DUIDs rather than device names in fstab? [yes]
```

The installer tells us that it sees one disk, device `sd0`. The installer must know which disk will hold the root partition. (With only a single disk this seems superfluous, but it becomes important if you system has we'll see an example with multiple disks, as discussed in "Custom Disk Layout" on page 49.) When you have only one disk, OpenBSD assumes that you'll use it. It also asks if you want to use the disk's DUID in the filesystem table rather than the device name. For reasons we'll discuss in Chapter 8, always answer **yes** to this.

The installer will now show you the MBR partition table.

```
Disk: sd0       geometry: 6527/ 255/ 63 [ 104857600 Sectors]
Offset: 0       Signature: 0xAA55
            Starting         Ending         LBA Info:
 #: id      C   H   S -      C   H   S [       start:        size ]
-------------------------------------------------------------------------------
 0: 00      0   0   0 -      0   0   0 [           0:           0 ] unused
 1: 00      0   0   0 -      0   0   0 [           0:           0 ] unused
 2: 00      0   0   0 -      0   0   0 [           0:           0 ] unused
 3: 00      0   0   0 -      0   0   0 [           0:           0 ] unused
Use (W)hole disk, use the (O)penBSD area, or (E)dit the MBR? [whole]
Setting OpenBSD MBR partition to whole sd0...done.
```

2. Of course, the US/Michigan time zone applies only to the four counties on the west end of the Upper Peninsula. But accepting the default wouldn't let me illustrate this, and if I have to make something up, it might as well be vaguely plausible.

The first line shows the detected hard drive geometry. This particular drive has 6527 cylinders, 255 heads, and 63 sectors per cylinder. If you compare this to the label on the physical drive, it almost certainly won't match (because hard drives lie). But note that this translated geometry has exactly the same number of sectors as shown in the hard drive documentation.

Beneath this line, you see the existing MBR partition table. The partitions are all zeroed out, which means that this drive has no partitions. We want only OpenBSD on this machine, so take the default and let OpenBSD swallow the whole drive.

Now it's time to consider your OpenBSD partitions.

```
The auto-allocated layout for sd0 is:
  #              size           offset  fstype [fsize bsize cpg]
❶ a:             1.0G               64  4.2BSD   2048 16384    1 # /
  b:             1.2G          2097216    swap
  c:            50.0G                0  unused
  d:             3.6G          4716480  4.2BSD   2048 16384    1 # /tmp
  e:             5.7G         12176320  4.2BSD   2048 16384    1 # /var
  f:             2.0G         24063040  4.2BSD   2048 16384    1 # /usr
  g:             1.0G         28257344  4.2BSD   2048 16384    1 # /usr/X11R6
  h:             6.3G         30354496  4.2BSD   2048 16384    1 # /usr/local
  i:             1.9G         43566400  4.2BSD   2048 16384    1 # /usr/src
  j:             2.0G         47467072  4.2BSD   2048 16384    1 # /usr/obj
  k:            25.4G         51661376  4.2BSD   2048 16384    1 # /home
❷ Use (A)uto layout, (E)dit auto layout, or create (C)ustom layout? [a]
❸ /dev/rsd0a: 1024.0MB in 2097152 sectors of 512 bytes
  6 cylinder groups of 202.47MB, 12958 blocks, 25984 inodes each
  ...
```

Our first partition at ❶ is a, which occupies 1GB and will be used as the root partition (/). On the installed system, this will be known as partition sd0a. Look down the list to see all of the standard partitions discussed in Chapter 2.

We could do custom disk partitioning at this point, but for our first installation, we'll use the defaults, as shown at ❷. The installer should then label the disk and ❸ create filesystems on all the partitions.

Choosing File Sets

Now that you have allocated disk space, let's put the operating system onto the disk. The installer starts by asking some basic questions about how to get the sets.

```
Let's install the sets!
Location of sets? (cd disk ftp http or 'done') [cd] ❶ ftp
HTTP/FTP proxy URL? (e.g. 'http://proxy:8080', or 'none') [none]
Server? (hostname, list#, 'done' or '?') [ftp5.usa.openbsd.org] ❷ ftp.lambdaserver.com
Server directory? [pub/OpenBSD/5.3/amd64]
Login? [anonymous]
```

Although I booted this system off a CD, I'm going to install the file sets via ❶ FTP. If my network needed to use a proxy to access the Internet, I would tell the installer.

While the installer will choose an FTP server for you at ❷, you can specify an FTP server that you know is close or fast. If you're installing a snapshot, give the file path to the desired snapshot on the FTP server. Finally, if this FTP server requires a username and password, enter it here.

At this point, the installer should log in to the FTP server, find all available file sets, and display them for your approval.

```
Select sets by entering a set name, a file name pattern or 'all'. De-select
sets by prepending a '-' to the set name, name pattern or 'all'. Selected
sets are labelled '[X]'.
    [X] bsd           [X] etc53.tgz     [X] xbase53.tgz   [X] xserv53.tgz
    [X] bsd.rd        [X] comp53.tgz    [X] xetc53.tgz
    [X] bsd.mp        [X] man53.tgz     [X] xshare53.tgz
    [X] base53.tgz    [X] game53.tgz    [X] xfont53.tgz
Set name(s)? (or 'abort' or 'done') [done]
```

I suggest you install everything, but you can choose to remove one or more sets.

For example, suppose you are building a firewall machine. Firewalls traditionally don't have compilers, documentation, or X. You can remove file sets by entering a minus sign (-) and the name of the file set.

```
Set name(s)? (or 'abort' or 'done') [done] ❶ -comp53.tgz -man53.tgz
    [X] bsd           [X] etc53.tgz     [X] xbase53.tgz   [X] xserv53.tgz
    [X] bsd.rd        [ ] comp53.tgz    [X] xetc53.tgz
    [X] bsd.mp        [ ] man53.tgz     [X] xshare53.tgz
    [X] base53.tgz    [X] game53.tgz    [X] xfont53.tgz
Set name(s)? (or 'abort' or 'done') [done]
```

This example removes the compiler and manual file sets at ❶. You can see that they're no longer selected in the list of file sets.

You can also use wildcards when selecting file sets. For example, here's how to remove all file sets beginning with an x:

```
Set name(s)? (or 'abort' or 'done') [done] -x*
    [X] bsd           [X] etc53.tgz     [ ] xbase53.tgz   [ ] xserv53.tgz
    [X] bsd.rd        [ ] comp53.tgz    [ ] xetc53.tgz
    [X] bsd.mp        [ ] man53.tgz     [ ] xshare53.tgz
    [X] base53.tgz    [X] game53.tgz    [ ] xfont53.tgz
Set name(s)? (or 'abort' or 'done') [done]
```

If you change your mind, you can add file sets back in by entering a plus (+) sign and the file set name. Here, I add back everything by using a wildcard (*):

```
Set name(s)? (or 'abort' or 'done') [done] *
    [X] bsd           [X] etc53.tgz     [X] xbase53.tgz   [X] xserv53.tgz
    [X] bsd.rd        [X] comp53.tgz    [X] xetc53.tgz
    [X] bsd.mp        [X] man53.tgz     [X] xshare53.tgz
    [X] base53.tgz    [X] game53.tgz    [X] xfont53.tgz
Set name(s)? (or 'abort' or 'done') [done]
```

Once you're ready, press ENTER to install the default or selected file sets.

After the installer unpacks all of the file sets on the hard drive, it will ask if you have more file sets to install.

```
Location of sets? (cd disk ftp http or 'done') [done]
```

If you have any custom file sets, you could install them at this point.

Finishing the Installation

After unpacking the file sets, the installer cleans up after itself and tells you it's finished with this message:

```
CONGRATULATIONS! Your OpenBSD install has been successfully completed!
To boot the new system, enter 'reboot' at the command prompt.
When you login to your new system the first time, please read your mail
using the 'mail' command.
```

Do as you're told and enter **`reboot`**, and then remove the CD if necessary. If you're content with a default installation, you can skip to Chapter 4 now.

Custom Disk Layout

If you have multiple hard disks in a system, or if you want a different partition layout than the default, you must manually edit your disk layout.

The installer partitions one disk at a time, and you can't easily bounce between multiple disks. To successfully use multiple disks, decide on your partitioning scheme *before* you start the installation, and write down exactly which partitions you want on which disks as specifically as possible.

My system has two 50GB hard disks. I plan to divide the disks like this:

Disk 1 1GB */*, 1.2GB swap, 5GB */tmp*, 1GB */usr/X11R6*, 2GB */usr/src*, 2GB */usr/obj*, and everything else */home*

Disk 2 1GB */altroot*, 1.2GB swap, 6GB */var*, 10GB */usr/local*, and everything else */var/postgresql*

This layout includes all of the standard OpenBSD partitions, plus a few additions: I've increased some partition sizes above the installer-generated defaults, and I've added an extra swap partition on the second hard drive. OpenBSD doesn't include a separate */var/postgresql* partition, but I've added one because I want my database data on its own partition. (We'll discuss the */altroot* partition in Chapter 9.)

The installer runs as usual until you get to the disk portion.

```
Available disks are: sd0 sd1.
Which one is the root disk? (or 'done') [sd0]
```

By default, the installer puts the root partition on the first hard drive, `sd0`. I'll use this disk for the root partition and use the entire disk for OpenBSD.

The installer then presents a list of automatically generated disklabel partitions. We don't want to use these partitions; we want to create our own from scratch.

```
Use (A)uto layout, (E)dit auto layout, or create (C)ustom layout? [a] c
```

We want a custom layout, so enter **c**.

The installer should now drop us to the `disklabel(8)` command prompt, indicated here by the > symbol:

```
You will now create an OpenBSD disklabel inside the OpenBSD MBR
partition. The disklabel defines how OpenBSD splits up the MBR partition
into OpenBSD partitions in which filesystems and swap space are created.
You must provide each filesystem's mountpoint in this program.

The offsets used in the disklabel are ABSOLUTE, i.e. relative to the
start of the disk, NOT the start of the OpenBSD MBR partition.

Label editor (enter '?' for help at any prompt)
>
```

We can now use the interactive disklabel editor to create OpenBSD partitions within the MBR partition, as discussed in the following sections.

Viewing Disklabels

The `p` command prints the partition's existing disklabel:

```
 > p
OpenBSD area: 64-104856255; size: 104856191; free: 32
#                size           offset  fstype [fsize bsize  cpg]
  a:          2104448               64  4.2BSD   2048 16384    1
  b:          2506143          2104512    swap
  c:        104857600                0  unused
  d:         10490432          4610656  4.2BSD   2048 16384    1
...
>
```

This is exactly the same information discussed in "Understanding Disklabels" on page 31. This hard drive previously had an OpenBSD installation, and the disklabel has those old partitions.

To display partition sizes in megabytes, enter **p m**:

```
> p m
OpenBSD area: 64-104856255; size: 51199.3M; free: 0.0M
#                size           offset  fstype [fsize bsize  cpg]
  a:          1027.6M               64  4.2BSD   2048 16384    1
  b:          1223.7M          2104512    swap
  c:         51200.0M                0  unused
  d:          5122.3M          4610656  4.2BSD   2048 16384    1
...
```

You can also display partition sizes in gigabytes by entering **p g**:

```
> p g
OpenBSD area: 64-104856255; size: 50.0G; free: 0.0G
#                size           offset  fstype [fsize bsize  cpg]
  a:             1.0G               64  4.2BSD   2048 16384    1
  b:             1.2G          2104512    swap
  c:            50.0G                0  unused
  d:             5.0G          4610656  4.2BSD   2048 16384    1
...
```

Choose the unit of measurement best suited to your disk.

Deleting Partitions

Use the `d` command to delete partitions:

```
> d
partition to delete: [] a
>
```

That's it. Tell `disklabel` to delete a partition on this disk, give it the partition letter, and it's gone. But beware: `disklabel` won't ask you to verify your choice, so be sure to choose the correct partition.

Erasing Existing Disklabels

You could manually delete all partitions, but it's much easier to zero out the existing disklabel with the `z` command:

```
> z
> p
OpenBSD area: 64-104856255; size: 104856191; free: 104856191
#                size           offset  fstype [fsize bsize  cpg]
  c:        104857600                0  unused
>
```

Here, we tell `disklabel` to erase the partition table with **z**, and then print the partition table with **p**. The output should be an empty disklabel, because the `c` disklabel partition represents the entire MBR partition. We can now create our desired partitions.

Creating Disklabel Partitions

This first disk needs the following partitions:

- 1GB */* (root)
- 1.2GB swap
- 5GB */tmp*
- 1GB */usr/X11R6*
- 2GB */usr/src*

- 2GB */usr/obj*
- Everything else */home*

By default, `disklabel` creates partitions in order. You can manually create partitions in any order you want, but you'll need to track sectors and cylinders in order to figure out where each partition should begin and end. I *strongly* recommend creating partitions in order and letting `disklabel` do the math.

Use the `a` command to create a partition beginning with `/`:

```
> a
❶ partition: [a]
❷ offset: [64]
❸ size: [104856191] 1g
❹ Rounding size to cylinder (16065 sectors): 2104451
❺ FS type: [4.2BSD]
❻ mount point: [none] /
❼ Rounding size to bsize (32 sectors): 2104448
>
```

By default, at ❶, `disklabel` offers the next free letter for your new partition. The first partition on the disk is `a`. Press ENTER to accept it.

The offset for a disklabel partition is the number of sectors from the beginning of the disk where the partition starts, not from the beginning of the MBR partition, which is the actual beginning of the disk. The first 63 sectors of a disk, numbers 0 through 62, contain the MBR. We could use sector 63, but OpenBSD starts on sector 64 to better align with memory cells in solid-state disks. At ❷, you can see that `disklabel` offers 64 as the default offset.

The size at ❸ is the number of sectors the partition uses. By default, `disklabel` offers all the remaining space on the disk, but I want a 1GB root partition. I could do the math to figure out how many sectors are in a gigabyte, but I'm lazy, so I use an abbreviation instead. The `disklabel` command recognizes the following abbreviations for sizes:

- `b` for bytes
- `c` for cylinders
- `k` for kilobytes
- `m` for megabytes
- `g` for gigabytes

All partitions must end on a cylinder boundary, so `disklabel` figures out the closest boundary and, at ❹, sizes my root partition to match. My root partition will be pretty close to 1GB.

The `FS type` at ❺ shows the filesystem used on this partition. For an OpenBSD disk, every data partition needs type `4.2BSD`. Your swap partition will be of type `swap`.

The mount point at ❻ is where you want this partition mounted. By default, `disklabel` doesn't assign a mount point because it can't guess what you want. Enter the partition's mount point.

Partitions must end on a cylinder boundary, but should end with a whole block for the filesystem. The `disklabel` command ❼ adjusts the partition size again, based on the standard block size of the filesystem.

Our next partition is swap space.

```
> a
❶ partition: [b]
❷ offset: [2104512]
❸ size: [102751743] 1.2g
❹ Rounding size to cylinder (16065 sectors): 2506143
❺ FS type: [swap]
>
```

The `disklabel` command assumes at ❶ that you're using the next partition letter, `b`. It automatically calculates the offset at ❷, which is the next free sector after the previous partition. I use decimal fractions at ❸ to set the size (I could alternatively enter `1200m`). The size at ❹ is rounded to the nearest cylinder boundary. Finally, `disklabel` knows that partition `b` is traditionally swap space, so it offers ❺ that as the default. Swap space doesn't need a mount point, and it doesn't have a block size.

We can create the remaining partitions in the same way. Creating the last partition, */home*, is even easier:

```
> a
partition: [h]
offset: [25575456]
❶ size: [79280799]
FS type: [4.2BSD]
mount point: [none] /home
Rounding size to bsize (32 sectors): 79280768
>
```

As you can see at ❶, we don't need to track how much empty disk space remains, because `disklabel` does that for us. Press ENTER to swallow it all. Now is your chance to leave empty space on your disk.

Now that you've created all your partitions, print the disklabel (with the `p` command, as described earlier in the chapter) to double-check your work.

Writing the New Disklabel

When you're satisfied with your partition scheme, enter `q` to write your disklabel to disk:

```
> q
Write new label?: [y] y
/dev/rsd0a: 1027.6MB in 2104448 sectors of 512 bytes
...
```

`disklabel` gives you one last chance to change your mind. Once you write a new disklabel, recovering any data on the disk becomes extremely

difficult, so be sure you backed up any vital data on this disk before starting the installation. (This is a good time to make sure that you didn't microwave your backup.)

Adding More Disks

After you partition your first disk, the installer offers you a chance to partition any other hard drives:

```
Available disks are: sd1.
Which one do you wish to initialize? (or 'done') [done] sd1
```

The default is to not partition any other disks. If you choose another disk, you'll need to create MBR partitions and then disklabel partitions.

Once all of your hard drives have been formatted, you'll return to installing the file sets.

Advanced Disklabel Commands

While the basic commands should suffice to partition your disk, `disklabel` supports a variety of advanced commands. We'll look at a few of them now.

Changing Basic Drive Parameters

Remember all that stuff at the top of the disklabel that shows the drive's basic physical characteristics? You can change all that, but it's almost never necessary. In fact, if you think doing this is a good way to solve a problem, you're probably on the wrong track.

If you enter `e`, `disklabel` walks you through each entry on the upper part of the disklabel. The existing values are presented as defaults, allowing you to quickly walk through the variables until you reach the one you want to change:

```
> e
Changing device parameters for /dev/rsd2c:
disk type: [SCSI]
label name: [Samsung HVX8812]
sectors/track: [63]
...
```

Edit this information at your own risk because you can render your disk unbootable or your partitions unusable by changing it! Changing the drive's physical description means you're lying to your computer, and computers go ballistic when you lie to them about their hardware.

Modifying Existing Partitions

The `m` command modifies existing partitions. The `disklabel` tool walks you through each of the values you entered when creating the disk, offering your original values as defaults and allowing you to change them. But most of the time, it's easier to just delete the partition and re-create it.

Entering Expert Mode

Expert mode gives the advanced user access to some rarely used options in `disklabel`. Most people don't need these and find them simply clutter. (It's not as if `disklabel` isn't complicated enough already.)

To access expert mode, use the `X` command. You won't immediately see all of the options available, but entering other commands will produce more options and more output.

Getting More Help

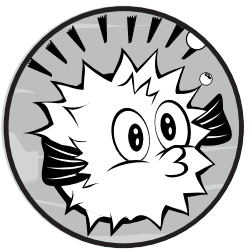
You can enter a single question mark (`?`) at the `disklabel` prompt for a brief list of all available commands. If you want more detailed help, the `M` command displays the `disklabel(8)` man page.

You've now installed OpenBSD. Let's see what to do next.

4

POST-INSTALL SETUP

Installation first,
now configure the software.
Server is ready.



You've installed OpenBSD and rebooted into a bare-bones system. Of course, a minimal Unix-like system is actually pretty boring. While it makes for a powerful foundation, it doesn't actually *do* much of anything.

To get you started, this chapter covers some of the basic steps you should take after installing OpenBSD to establish a firm platform for later work. We'll jump right into basic tasks such as correcting the time zone, setting a default gateway, and setting an email alias for the root account.

But a bit of forewarning: You can change a surprising amount of OpenBSD's configuration when the system is up and running. You can reconfigure the hostname, network configuration, and time and date, as well as stop, start, or reset daemons as you see fit. But just because you can doesn't mean that you should.

While playing arbitrarily with a desktop or laptop machine might be fine, if you're running a server, you should test your configuration by rebooting. Make sure that the system boots exactly as desired before adding services.

If you find that OpenBSD boots fine but you must poke the network card before it works, fix that before proceeding.

NOTE *The `afterboot(8)` man page has up-to-date advice for systems administrators who have just installed their first OpenBSD system. Much of this advice overlaps material that we'll cover in this book (in a much more exciting fashion, I'm sure), while some of it applies only to specific use cases. Read the `afterboot` documentation on your system for the very latest information.*

All of the tasks in this chapter must be performed as root. We'll discuss creating additional users in Chapter 6 and ways to avoid using the root account in Chapter 7, but you don't need to do that on a newly installed system. Configure the system properly before you start letting people log on, or you'll wish you had.[1]

First Steps

The first two things you should do after installing a new system are check for any operating system patches or errata and change the root administrative password. These steps are critical, so don't skip them.

Checking the System Errata

Believe it or not, OpenBSD isn't perfect. Releases sometimes have bugs. Some of these are serious problems; others not so much.

When the OpenBSD team learns of a serious problem with a release, it issues an errata list, and whenever you build a new server, you should check the errata list at *http://www.OpenBSD.org/errata.html*. Critical errata are also announced on *security-announce@OpenBSD.org*, so if you're on that mailing list, you'll get notifications of new errata. You'll also see errata notices on *http://www.undeadly.org/*.

Errata won't always affect your use case. For example, as I write this, OpenBSD 5.0 has one errata notice: a problem in the BIND name server. If this server won't run BIND, don't worry about this errata. If you're building a name server, however, you need this information before going into production.

If you're in doubt, correct your system as recommended in the errata, which may require building one or more parts of OpenBSD from source. (I'll discuss errata and building OpenBSD at length in Chapter 20.)

Setting the Root Password

You needed to choose a root password during installation. To change it, use the `passwd(1)` command. Of course, you must be root to change root's password.

1. Saying "Sorry about the timestamps errors in your vital data, but I hadn't set the system clock yet" is roughly equivalent to saying "I don't care about you or your data." If you feel that way, I'm not going to argue, but at least have the confidence to tell the user what you really think.

Software Configuration

When the OpenBSD kernel finishes its initial system setup and hands control of the system over to userland, `init(8)` runs the shell script */etc/rc*. This script starts all of the programs integrated with the system and performs general system configuration, such as configuring network interfaces and starting server software. To enable, disable, or otherwise configure integrated software, modify the files */etc/rc.conf* and */etc/rc.conf.local*. (I'll cover the OpenBSD boot process in detail in Chapter 5, but for now, this section will get you started.)

The files *rc.conf* and *rc.conf.local* contain shell script variable assignments that control what */etc/rc* runs and the command-line options for the various programs. Keep in mind that any entries in *rc.conf.local* override *rc.conf* statements. Most variable assignments have three legitimate values: an uppercase `NO`, command-line flags in quotation marks (`"-D"`), or double quotes (`""`), which are equivalent to empty. Each variable looks something like this entry from *rc.conf*:

```
ntpd_flags=NO          # for normal use: ""
```

The variable `ntpd_flags` controls the command-line flags that */etc/rc* uses when starting `ntpd(8)`.

A `NO` disables this particular piece of functionality. In the preceding example, the NTP daemon `ntpd(8)` is disabled.

If the variable is empty, */etc/rc* starts the program without any command-line arguments. For example, this `ntpd_flags` entry means that `ntpd` is to be started without any arguments.

```
ntpd_flags=""
```

Anything within quotes is used as a command-line argument to the program. (If a program has typical default flags, they'll usually appear in *rc.conf*.) The following example assigns the variable `ntpd_flags` the value `-s`. When the system boots, rather than running `ntpd`, it will run `ntpd -s`:

```
ntpd_flags="-s"
```

Some variables have additional possible values. For example, the PF packet filter (see Chapter 21) is enabled with a `YES`. To enable the NFS automounter daemon, you'll need to use a path to the master map. (If you don't know what the automounter is, that's fine—not many do these days.) Just realize that weird values for *rc.conf* variables do exist. You'll see these values listed in *rc.conf*.

NOTE *OpenBSD defaults appear in* /etc/rc.conf*, but do not edit this file! This is a core system file, and will be replaced during an upgrade. Put your local changes in* /etc/rc.conf.local*. Entries in* rc.conf.local *will override the defaults in* rc.conf.

Time and Date

Correct system time is not only a convenience, but also a security issue, because many attacks rely on changing the system clock. However, if your system clock is wrong to start with, you won't notice a change. Without coherent time across all your servers, you'll never be able to correlate your logs when troubleshooting. What's the solution? Fix your time settings before you do anything else. Correcting the time requires both setting a time zone and the clock.

Setting the Time Zone

The installer tries really hard to guess your time zone, using geolocation tricks and a script at the OpenBSD website. If these didn't work for you, or if you weren't on the Internet when you installed OpenBSD, or if your company policy says that all servers will run in time zone *X*, fix your time zone before anyone notices.

The directory */usr/share/zoneinfo* contains all of the time zones, as well as several subdirectories for countries or continents with various time zones. For example, Western Siberia runs on Omsk time (found in the file */usr/share/zoneinfo/Asia/Omsk*). Presumably, you have some idea of your local time zone and where it might be filed.

To set the system time zone, create a symbolic link to it from */etc/localtime*. Use `date(1)` to make sure that the time zone has been set correctly:

```
# ln -fs /usr/share/zoneinfo/Asia/Omsk /etc/localtime
# date
Thu Mar 14 06:02:56 OMST 2013
```

OpenBSD also supports POSIX time zones found in */usr/share/zoneinfo/Etc*. POSIX time zones have their own rules. Do not use them unless you are absolutely sure you understand them. (Hint: You don't.)

Setting the Date and Time

Now that you have set a time zone, set the correct time and date. OpenBSD includes OpenNTPD, a BSD-licensed simplified NTP daemon. If at all possible, use `ntpd(8)` to manage the time. If you can't access NTP servers (say, if you're on a private network without them), set up your own. And if you can't set up time servers, set the system time manually.

Setting the Time with ntpd(8)

Configure OpenNTPD in */etc/ntpd.conf*. The syntax should be familiar to you if you've managed any other NTP daemon.

For basic time, you need time servers, ideally three or more. If you don't have local time servers, use publicly accessible time servers, such as the hosts available at *http://pool.ntp.org/*.

List your servers in */etc/ntpd.conf*:

```
servers pool.ntp.org
```

Then enable `ntpd` in */etc/rc.conf.local*:

```
ntpd_flags=
```

By default, `ntpd` slowly adjusts system time by skewing the system clock. If the system time is a few seconds off, slow adjustment will usually suffice, but if it's off by minutes or more, have `ntpd` correct the system time on startup and then adjust the time as needed. To enable time correction at startup, use the `-s` flag:

```
ntpd_flags="-s"
```

Time skews most badly on heavily used hardware, lousy hardware, and virtual machines.

Setting the Date Manually

To set the date and time manually, use `date(1)`. First, make sure that you know the current year, month, day of the month, and time (in 24-hour format). Then set the date and time using this format:

```
# date YYYYmmDDhhMM
```

For example, to set the date to February 3, 2013 and the time to 1:17 PM, run this:

```
# date 201302031317
Sun Feb  3 13:17:00 GMT 2013
```

That said, `date(1)` will not correct your clock on an ongoing basis, and on some hardware with poor clocks, the time will slowly skew. A virtual machine on heavily loaded hardware will almost certainly lose time. Use NTP to deal with that.

Hostname

Set the system's hostname in */etc/myname*. My test system is called *caddis.blackhelicopters.org*.

```
$ cat /etc/myname
caddis.blackhelicopters.org
```

To change the hostname, edit */etc/myname*. The new hostname takes effect after the next reboot.

To change the hostname until the next boot, use `hostname -s` and the new hostname, like this:

```
# hostname -s treble.blackhelicopters.org
# hostname
treble.blackhelicopters.org
```

You can edit */etc/myname* and run `hostname -s` to make a change take effect immediately and persist after the next boot.

Networking

If you installed OpenBSD over the Internet, at least one Ethernet card should be configured and working. But if you installed from CD, you didn't need to configure the network to install OpenBSD. If you installed OpenBSD on one network and want to move the machine to another network, you'll need to reconfigure the network on your system.

I cover network principles in Chapter 11 and configuration in Chapter 12, but this brief entry will attach valid network addresses to your system, install a default route, and get DNS resolution working. If you're not sure what to do here, don't do anything until you read the later chapters.

To make network changes take effect, you can either reboot or run the network startup script */etc/netstart*, like this:

```
# sh /etc/netstart
```

To configure only one interface, give the interface's name as an argument:

```
# sh /etc/netstart em0
```

Again, if your system is a server, reboot before you declare the server ready for production.

Configuring Ethernet Interfaces

For a complete list of network interfaces recognized by your host, run `ifconfig(8)`. You should see a bunch of entries, like this:

```
❶ em0: flags=8843<UP,BROADCAST,RUNNING,SIMPLEX,MULTICAST> mtu 1500
        lladdr 00:0c:29:d2:37:be
        priority: 0
        groups: egress
        media: Ethernet autoselect (1000baseT full-duplex,master)
        status: active
❷       inet 192.0.2.36 netmask 0xfffffe0 broadcast 192.0.2.63
        inet6 fe80::20c:29ff:fed2:37be%em0 prefixlen 64 scopeid 0x1
```

Every OpenBSD system comes with the default interfaces `lo0` (loopback), `enc0` (encapsulating interface), and `pflog0` (PF logging, discussed in Chapter 21). These are virtual interfaces that don't actually attach your system to the local Ethernet. Other interfaces will be physical network ports, like the Ethernet interface `em0` at ❶.

Every Ethernet card has its own configuration file, */etc/hostname* .interfacename. The `em0` interface in the example is configured in the file */etc/hostname.em0*. If the file doesn't exist, create it.

At ❷, you see the IP address. The format of static IP addresses depends on the IP version in use.

Static IP Addresses

For IP version 4 (IPv4) addresses, the common format is as follows:

```
inet ipaddress netmask broadcastaddress options
```

This format includes these elements:

- The `inet` keyword indicates this is an IPv4 address.
- The IP address (*`ipaddress`*) appears in standard dotted-quad notation.
- The *`netmask`* can appear in dotted-quad format (255.255.255.224) or in hexadecimal (0xffffffe0).
- The broadcast address (*`broadcastaddress`*) gives you the option to hardcode the broadcast address on this network. If you leave this blank or use the word `NONE`, OpenBSD computes the correct broadcast address from the IP address and netmask given earlier. If you use `ifconfig(8)` options, you must use the word `NONE` to provide spacing. (You can also provide the netmask in "slash" notation directly after the IP address, as in `192.0.2.2/24`.)
- The *`options`* space is where you should put any specific `ifconfig(8)` commands needed for this interface, like hardcoded speed or duplex. (See Chapters 11 and 12 for examples, and the `hostname.if(5)` man page for the full details.)

When working with IP version 6 (IPv6) addresses, the format is as follows:

```
inet6 address prefixlength options
```

This format includes these elements:

- The `inet6` keyword tells the system that this is an IPv6 address.
- The *`address`* space is for the IPv6 address, without a prefix length.
- The prefix length (*`prefixlength`*) appears separately, without a slash.
- As with IPv4, you can use `ifconfig(8)` *`options`* as needed.

Even a very simple configuration will get a machine on the network. The following configures `em0` with the IPv4 address 192.0.2.2 and netmask 255.255.255.224. It also has the IPv6 address of 2001:DB8:7700::2/64.

```
$ cat /etc/hostname.em0
inet 192.0.2.2 255.255.255.224
inet6 2001:DB8::2 64
```

Anything that doesn't follow these IPv4 and IPv6 formats is passed directly to `ifconfig`. However, if you don't know if a particular configuration will work, try it at the command line first.

Dynamic Configuration

When performing dynamic configuration, if the machine is an IPv4 DHCP client, use the string `dhcp` in *hostname.if*. For IPv6 autoconfiguration, use the string `rtsol`, by itself on a single line, to tell OpenBSD to use the `rtsol(8)` IPv6 autoconfiguration program.

```
$ cat /etc/hostname.em0
dhcp
rtsol
```

In order for IPv6 autoconfiguration to work, you must disable IPv6 routing with these two entries in */etc/sysctl.conf*:

```
net.inet6.ip6.forwarding=0
net.inet6.ip6.accept_rtadv=1
```

These values already exist in */etc/sysctl.conf*, but they're commented out.

Setting a Default Gateway

To set either the IPv4 or IPv6 default gateway, place the gateway IP address on a single line in */etc/mygate*, with no other entries in the file. You can configure default gateways for both protocols, each on its own line. The change should take place on your next reboot, or use `route(8)` to manually change the default gateway. The following sets both an IPv4 and an IPv6 default gateway.

```
192.0.2.1
2001:DB8::1
```

Dynamic configuration requires setting a default route. If your */etc/hostname.if* file contains a dynamic configuration statement, */etc/mygate* will not be used for that IP protocol.

Setting Name Service Servers

If you want to contact other machines by hostname, you need to set Domain Name Service (DNS) servers.

Chapter 12 covers DNS resolution in detail, but */etc/resolv.conf* contains the basic client settings. Its first line defines the local domain with the `domain` keyword and a domain name. Name servers appear on subsequent lines, defined with the keyword `nameserver` and an IP address, as shown here:

```
domain blackhelicopters.org
nameserver 192.0.2.1
nameserver 192.0.2.3
```

Mail Aliases and Status Mail

Every OpenBSD system runs maintenance tasks daily, weekly, and monthly, and sends email messages with the results to the local root account. I discuss these maintenance jobs in Chapter 15. If you have more than a couple of servers, you should probably forward these email messages to a single central account.

The OpenBSD system redirects email messages addressed to local users to other users or remote email addresses, as configured in */etc/mail/aliases*. Here are a few entries:

```
...
# Basic system aliases -- these MUST be present
MAILER-DAEMON: postmaster
postmaster: root
...
```

The original recipient appears on the left, followed by a colon, and a list of people to forward the messages to. If you have multiple recipients separate them with commas. Mail addressed to MAILER-DAEMON on the local system is forwarded to the postmaster account, which, in turn, is forwarded to the root account.

You should create an alias for root, directing mail sent to root to your systems administration team:

```
root: mwlucas@bigcompany.com, sysadminstaff@bigcompany.com
```

After changing */etc/mail/aliases*, run `newaliases(8)` to update email forwarding.

Keyboard Mapping

OpenBSD tries to guess your correct keyboard mapping. USB keyboards have mechanisms to declare their country code. If autodetection doesn't work, you can change the keyboard layout with `kbd(8)`, and set it at boot with */etc/kbdtype*.

Before touching the keyboard layout, use `kbd` to find your current keyboard map. `kbd -l` lists the more than 100 keyboard encodings OpenBSD supports. Browse the list for your supported keyboard layout, or use `grep` to reduce the list. I'm a Dvorak user, so I search for it like this:

```
# kbd -l | grep dvorak
fr.dvorak
us.dvorak
...
```

My preferred layout is `us.dvorak`. I enter **`us.dvorak`** in */etc/kbdtype*, and after my next reboot, the console should use the Dvorak layout.

To set the keymap immediately to Dvorak, I could use `kbd`:

```
# kbd us.dvorak
kbd: keyboard mapping set to us.dvorak
```

Read more about changing the console in Chapter 17.

Installing Ports and Source Code

If you intend to build your own OpenBSD releases or heavily customize OpenBSD, you'll need the operating system source code (see Chapters 18, 19, and 20). If you intend to build your own packages (covered in Chapter 13), you'll need the ports tree. Both the ports tree and the system source code are specific to an OpenBSD release, and if you need them, install them when you install the system.

The ports tree is the compressed file *ports.tar.gz* in the release directory where you stored OpenBSD—either on the mirror site or the CD. All architectures share one ports tree. Download this file to your home directory and extract it in */usr*.

```
# cd /usr/
# tar -xzvf $HOME/ports.tar.gz
```

This extracts the ports tree to */usr/ports*.

The system source code is found in three files in the release directory: the kernel source code *sys.tar.gz* (see Chapter 19), userland source code *src.tar.gz* (see Chapter 20), and the Xenocara *xenocara.tar.gz* (see Chapter 20). Extract Xenocara to */usr*, and the others to */usr/src*.

Booting to a Graphic Console

OpenBSD boots into a text console by default. To boot into an X-based graphic console, use the `xdm(1)` command in */etc/rc.conf.local*. The default `xdm` setting is `NO`, so by changing it to empty quotes you enable it.

```
xdm_flags=""
```

If you need a more complicated `xdm` setup, put any command-line arguments inside the quotes.

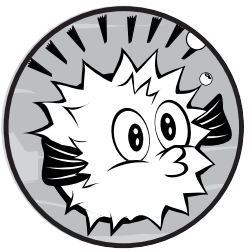
Onward!

Armed with the information in this chapter, you should be able to get your system on your local network and make it at least minimally comfortable to work with. Next, let's take a look at the OpenBSD boot process.

5

THE BOOT PROCESS

Single-user mode
unscheduled in the nighttime?
Something just went "boom"!



In order to properly manage any computing platform, you must understand the boot process. Many systems administration tasks cannot be done while the system is running. OpenBSD specifically requires that certain tasks be done before the boot process has completed. And, of course, on any operating system, sometimes a process starting up prevents the system from completing its boot. The only way to fix these problems is to interrupt the boot partway through.

First, we'll look at the key to OpenBSD's booting process: the boot loader. Then we'll move on to single-user mode, and finally multiuser startup. You can perform useful work at any of these stages.

I recommend playing with the OpenBSD boot process on a test machine *before* one of your machines won't boot. That way, when something breaks in the wee hours of the morning, you can spend your time fixing the problem instead of fumbling around with unfamiliar commands.

Power-On and the Boot Loader

In general, when a PC-style computer first boots, it starts the BIOS. The BIOS is a small piece of software that figures out things like which drives are attached and what they're attached to, what sort of CPU is installed, and how much memory is available. After getting that information, the BIOS loads a minimal boot loader from some kind of storage device.[1]

The *boot loader* is a small program that handles initial system configuration and boots the kernel. It finds and starts the kernel, which in turn detects hardware, attaches device drivers, and performs other core setup. Finally, the kernel calls `init(8)`, which starts processes and enables user programs, network interfaces, server software, and so on.

While most of this process cannot be managed—no one actually configures `init`!—there's plenty you can do before the system finishes booting and dumps you at the login screen.

The OpenBSD boot loader lets you interrupt the boot process, configure the system before it boots, adjust kernel settings, and even boot an alternate kernel.

When the hardware hands control of the boot process over to the OpenBSD partition, you'll see the boot loader prompt, which looks something like this:

```
>> OpenBSD/amd64 BOOT 3.18
boot>
```

The boot loader's main purpose is to find the kernel, load it into memory, and start it. Because it runs before the kernel starts, the boot loader can pass instructions to the kernel itself.

Here are some of the things you can do before booting is complete:

Use built-in help

Use the `help` function to print a brief list of commands that the boot loader supports.

```
boot> help
commands: # boot echo env help ls machine reboot set stty time
machine: boot diskinfo memory
```

Delay the boot process

By default, the loader waits five seconds for instructions, and then boots the kernel. To pause the boot at the prompt, press the spacebar.

Set the boot timeout

To set a new boot idle timeout, specify a number of seconds with the `set timeout` command.

1. On i386 and amd64 systems, this is where the MBR comes in.

```
boot> set timeout 10
```

After the boot prompt is idle for 10 seconds, the system should boot.

Boot the system

If you've paused the boot process, the system won't boot until you tell it to. When you're ready to boot, use the `boot` command:

```
> boot
```

We'll use various permutations of `boot` to configure the kernel, boot single-user mode, and so on. I'll cover other boot commands in the appropriate sections. For full details on what you can do at the boot loader prompt, read the `boot(8)` man page.

Booting in Single-User Mode

Single-user mode is the earliest point when OpenBSD can give you a Unix-style shell prompt. At this point, the kernel has probed all the hardware, attached drivers to all the hardware that it's going to acknowledge, and started `init`. The system hasn't mounted any filesystems except for the root partition, which is mounted in read-only mode. The network isn't started, no services are running, security is not implemented, and filesystem permissions are ignored.

To boot OpenBSD in single-user mode, enter **`boot -s`** at the loader prompt.

```
boot> boot -s
```

Why would you want to boot into single-user mode? If your computer has a problem that is preventing it from booting, you should be able to access single-user mode and fix the problem. Suppose a failed disk is preventing the system from booting during a multiuser boot, or you changed your terminal settings in */etc/ttys* and now you can't log on to the system. Or maybe you put a daft setting in *rc.conf.local*, and the boot process hangs because it's trying to do something impossible. At times like these, single-user mode is your best friend.

Also, some system administration tasks, such as clearing filesystem flags (see Chapter 8), can be done only in single-user mode.

Mounting Disks in Single-User Mode

Usually, you should have a fully functional filesystem before doing anything in single-user mode. If your system crashed, be sure to check the integrity of your filesystems before mounting them:

```
# fsck -p
/dev/sd0a (e4bf0318329fe596.a): file system is clean; not checking
```

```
/dev/sd0h (e4bf0318329fe596.h): file system is clean; not checking
...
# mount -a
```

`fsck` and `mount` have many more options. We'll cover them in more detail in Chapter 8.

Once you've mounted all of your filesystems, all usual command-line software should be available. You should be able to edit configuration files, start and stop programs, and generally do whatever you like to the system (including destroy it).

Starting the Network in Single-User Mode

Use the shell script */etc/netstart* to configure the network while in single-user mode. (You could run all the appropriate commands by hand, but */etc/netstart* will read your system's configuration files and do the grunt work for you.) You must explicitly run this script through `sh`:

```
# sh /etc/netstart
```

If you're booting into single-user mode because of network problems, this script will conveniently reproduce the issue for you.

Booting an Alternate Kernel

As we'll cover in tedious detail in Chapter 18, you *can* configure the OpenBSD kernel, but before you do so, be sure that you can boot alternate kernels. You'll need to be able to boot a different kernel if, say, you hose your filesystem so badly that it won't even boot to single-user mode, and you need to recover using the installation kernel.

Booting a Different Kernel File

An OpenBSD installation includes three kernels out of the box: the single-processor kernel */bsd*, the multiprocessor kernel */bsd.mp*, and the upgrade and install kernel */bsd.rd*. (If your machine has multiple processors, the installer renames */bsd* to */bsd.sp* and */bsd.mp* to */bsd*.)

To boot a nonstandard kernel, first reboot and interrupt the boot process at the boot loader prompt. Run `boot`, and give the full path to the kernel you want to boot:

```
boot> boot /bsd.rd
```

This will start the system using your chosen kernel. You can use other boot options as well, such as booting the alternate kernel in single-user mode:

```
boot> boot -s /bsd.sp
```

This will let you recover from a bad kernel, try a new kernel, or anything in between.

Booting from an Alternate Hard Disk

Suppose you've really fouled everything up beyond all recognition, and you don't have a usable kernel on your root partition. Fortunately, if you have a usable kernel on a different hard drive, you can boot from that. (Usually, this kernel lives on an alternate root partition, */altroot*, as discussed in Chapter 8.) In this section, I'll break the task of booting from that alternate kernel into a few steps: finding the hard disk with the partition, finding the partition with the file, and booting the right file on that partition.

Finding the Disk

Once you're familiar with OpenBSD, you may begin to think of the hard drives in your system by their device names, such as */dev/sd0*, */dev/wd1*, and so on. Unfortunately, those are the kernel's names for the disks; the boot loader recognizes only the BIOS's disk names.

To ask the boot loader about disks, use the `machine diskinfo` command:

```
boot> machine diskinfo
Disk    BIOS#   Type    Cyls    Heads   Secs    Flags   Checksum
fd0     0x0     *none*  80      2       18      0x4     0x0
hd0     0x80    label   1024    255     63      0x2     0x51db843d
hd1     0x81    label   1024    255     63      0x2     0x9329b723
hd2     0x82    label   1024    255     63      0x2     0xcfadb343
boot>
```

Here, the boot loader has found four disk devices. The first, `fd0`, is a floppy disk drive. This drive might or might not have a disk in it, but whatever it has, it's almost certainly not your alternate kernel. (It might be an installation disk, though, so don't automatically rule out using it for disaster recovery.)

The other three devices—`hd0`, `hd1`, and `hd2`—are hard disks. The first, `hd0`, is the default system boot disk. If you can't boot from that disk, you need to find the hard disk that contains your kernel.

Finding the Partition

Vague stirrings of memory in this output lead me to think that `hd2` might be the disk that holds my backup root partition. To try it, tell the loader that disk partition `hd2a` is the new root partition:

```
boot> set device hd2a
```

Before trying to boot from this partition, look at its contents:

```
boot> ls
stat(hd2a:/.): Invalid argument
boot>
```

Apparently, disk `hd2` has no partition `a`. After service is restored, I'll take this disk out behind my garage and beat its weakness out of it. For now, let's try the only remaining disk, `hd1`.

```
boot> set device hd1a
boot> ls
drwxr-xr-x 0,0  512     .
drwxr-xr-x 0,0  512     ..
drwxr-xr-x 0,0  512     altroot
drwxr-xr-x 0,0  512     home
drwxr-xr-x 0,0  512     tmp
...
```

This looks like an actual root partition (`altroot` offers a hint).

Booting the Kernel

At this point, we could boot a different kernel, but we'll just boot the */bsd* kernel on this partition in single-user mode, because the filesystem table would have the incorrect entry for the root filesystem, which would mess up all sorts of stuff.

```
boot> boot -s
booting hd1a:/bsd: 5669864+1601484+935608+0+617568 [89+499848+323884]=0xd351b8
...
```

Alternatively, you could give the device name at the boot prompt:

```
boot> boot -s hd1a:/bsd
```

As a general rule, you should mount the actual root partition on */mnt*, make the necessary changes for normal operation, and reboot into the proper root partition. You could also boot the */bsd.rd* kernel, giving you a cleaner boot at the cost of having fewer tools available.

Making Boot Loader Settings Permanent

To make boot loader options permanent, edit */etc/boot.conf*. The boot loader reads and runs entries from this file before giving you the `boot>` prompt, which means you can use it to automatically run boot loader commands every time your computer boots. (Although if you would rather sit at your computer and enter your settings every time you reboot, don't let me stop you.)

Any command you might give at the loader prompt is a valid *boot.conf* entry. For example, if the default boot speed is too slow for your liking, you can set your boot timeout to two seconds by adding this line to *boot.conf*:

```
set timeout 2
```

You can also tell the system to boot a different kernel with the correct *boot.conf* command.

```
set image /bsd.mp
```

By far, *boot.conf* is most often used to configure a serial console.

Serial Consoles

All of these nifty boot functions let you do useful stuff when your system is in trouble, but how can you use them when your computer isn't right in front of you? If your computer is in a data center on the other side of the country, or sitting in the basement behind the last decade of payroll records, a serial console will make your life far more pleasant.

A hardware serial console allows you to run a serial cable between a computer and a terminal server (on another computer) to access BIOS messages and operating system boot and startup messages which simplifies managing remote systems. Serial consoles are invaluable when debugging system crashes, too; error messages come over the serial port, where you can easily capture them.[2]

True UNIX hardware has serial console capabilities, as does most server-grade i386 and amd64 hardware. Most desktop-grade hardware, however, does not. But fortunately, even if you don't have a hardware serial console, you can access all of OpenBSD's startup messages with a serial port and a software serial console. While OpenBSD's software serial console won't give you access to the hardware BIOS, it will let you interface with the boot loader and remotely access the system console, even when the network is down.

Other Platform Serial Consoles

Every hardware platform has its own standards for serial consoles. If you're running a less common platform, check your hardware's documentation.

If your hardware supports a real serial console, you should usually configure it in the BIOS. OpenBSD supports whatever the hardware supports, so your Sparc64 hardware will support OpenBSD's serial console just as well as it supports any other operating system's serial console.

Serial Console Physical Setup

A serial console requires a null modem cable, which you should be able to get from any computer store or an online vendor. While gold-plated cables aren't worth the money, don't buy the cheapest cable you can find either. If you have an emergency and need the serial console right *now*, you won't be in the mood to deal with a defective cable.

2. Granted, a remote keyboard-video-mouse (KVM) system can give you all of this, but very few KVM applications let you copy and paste text from the remote console. That means you'll need to copy error messages by hand.

Plug one end of the null modem cable into your OpenBSD machine's first serial port. (The serial console is supported on only the first serial port, or `com0` on i386 and amd64 hardware.) Plug the null modem cable's other end into an open serial port on another system. (For simplicity's sake, use either another OpenBSD or Unix-like system.)

If you have two OpenBSD machines at a remote location and you want to use serial consoles on both, you can have each machine act as the console client for the other. Attach the first serial port on each server to the second serial port on the other. If you have three machines, you can daisy-chain them in a loop. If you have four or more machines, pick up a used terminal server from your favorite auction site.

You can also use two DB9-to-RJ45 converters, one standard and one crossover, which will allow you to run your console connections over a standard CAT5 cable. If you have a lights-out data center where human beings are forbidden unless they are installing or removing equipment, you can stretch your serial console cables about 12 meters, which should reach into your warm room. (Most modern data facilities are better equipped to handle CAT5 cables than serial cables.)

Serial Console Configuration

Now that you have the console physically ready, the next step is to configure your client to access the serial console. Then you can set up the serial console.

Configuring the Serial Console Client

The following are the default settings for an OpenBSD i386 or amd64 system:

- 9600 baud
- 8 bits
- No parity
- 1 stop bit

Enter these values into any terminal emulator on the client computer, and the serial console should Just Work. You can find terminal emulators for Microsoft platforms (I recommend PuTTY), OS X, and just about any other operating system.

OpenBSD includes the terminal emulator `tip(1)`, which reads its configuration from */etc/remote*. The configuration `ttyOO` in */etc/remote* matches the default OpenBSD serial console configuration for i386 and amd64 systems (as well as several other platforms). If you've attached your null modem cable to the first serial port on the client, connect with this command:

```
# tip ttyOO
connected
```

If it doesn't say `connected`, your serial client is misconfigured. Fix your client before enabling your serial console on the server. You want your serial client ready before configuring the console.

Setting Up the Serial Console

OpenBSD normally uses the local physical keyboard, video, and mouse as the console, but it can also use the first serial port as a serial console.

To set the console, use the boot loader. You must know the loader's device name for your preferred console: `com0` for the first serial port or `pc0` for the physically attached video and keyboard.

The first time you try to use a serial console, use a local test machine. Set up your client beforehand and start your terminal emulator, and then boot your test machine. At the boot loader prompt, enter this command:

```
boot> set tty com0
```

Your server's monitor and keyboard should stop responding, and if you've set up everything correctly, you should see the boot loader prompt in your terminal emulator.

To switch back to the physical console, tell the boot loader to use the `pc0` device:

```
boot> set tty pc0
```

Poof! The server's keyboard and monitor should work again.

To have your machine use the serial console at every boot, add this statement in */etc/boot.conf*:

```
set tty com0
```

Be sure to test your serial console after the machine is installed in its permanent location, and always screw the serial cables to the server. A loose serial cable provides only a comforting illusion that betrays you when it will hurt the most.

Testing the Serial Configuration

After configuring your serial console, return to your serial client and press ENTER. You should see something like this:

```
OpenBSD/amd64 (caddis.blackhelicopters.org) (tty00)

login:
```

Changing the Serial Console Speed

Newer serial ports (meaning anything made within the past 10 years) can run at speeds far above 9600 baud. I have servers with serial consoles that

run only at 115,200 baud. The BIOS messages display at 115,200 baud, but then the OpenBSD console runs at 9600 baud. My client displays one or the other as gibberish. (A lot of OpenBSD folks think that anything that won't do serial at 9600 baud is broken, but you won't always have control over the hardware you work with.)

To use these ports, I can either change my connection speed in my serial console client when switching between the BIOS messages and the OpenBSD messages, or change the speed of my OpenBSD console to match the hardware.

At the boot loader, tell the serial console to run at 115,200 baud:

```
boot> stty com0 115200
boot> set tty com0
```

If these settings work, copy them to */etc/boot.conf*.

Now configure your serial client. Modify `tip` to use the higher speed. First, find the entry for `tty00` in */etc/remote*:

```
tty00|For hp300,i386,mac68k,macppc,vax:\
        :dv=/dev/tty00:tc=direct:tc=unixhost:
```

But don't modify this entry! We'll use it to illustrate the style of */etc/remote* entries.

NOTE /etc/remote *is designed much like a* `termcap(5)` *database. If you ever need to write your own* `termcap` *entries from scratch, you're living your life wrong. But you can recognize the contents and modify existing entries without much pain. If you really want to learn everything about these entries, read the* `remote(5)` *man page.*

Backslashes (`\`) in this entry mean "continued on the next line." Colons separate fields. Each line after the first must start with a colon, and each field is a key/value pair.

Now, to create a console entry that runs at 115,200 baud, use the following:

```
console:br#115200:tc=tty00:
```

The first field in an */etc/remote* entry is the name, and every entry must have a unique name. I named this entry `console`. The second field is the `br` value. According to `remote(5)`, `br` stands for bit rate. I've set the bit rate to 115,200 baud. The third field is `tc`, for "table continues," which is equal to `tty00`. This means that the description of this entry continues in entry `tty00`.

Taken as a whole, this entry means "copy the `tty00` entry, and add a bit rate of 115,200."

Changing the Client Serial Port

If you have two OpenBSD machines, each sending its serial console out its first serial port to the other machine's second serial port, you must tell `tip`

to connect to the second serial port. The command `tip tty00` doesn't actually connect to the serial port named `tty00`—it connects to a port defined by the */etc/remote* entry named `tty00`. That means that you can't run, say, `tip tty03` and connect to serial port tty03 unless you have an */etc/remote* entry named `tty03`. By default, there isn't one, but you can define one easily, as follows:

```
tty03:dv=/dev/tty03:tc=tty00:
```

This entry is named `tty01`. The `dv` setting tells */etc/remote* the physical device to use. Other than this, all settings are copied from the entry called `tty00`.

With these examples, you should be able to use OpenBSD's `tip` to connect to almost any serial console.

Serial Logins

The serial console lets you interact with the boot process. Once your machine is fully multiuser, however, a default serial console will not let you actually log in to OpenBSD. In multiuser mode, OpenBSD uses `getty(8)` to initialize terminals and handle logins, and in order to log in to your machine over a serial port, you will need to tell `getty` to take charge of the serial line by configuring */etc/ttys*.

We'll discuss */etc/ttys* further in Chapter 14, but for now, here's how to allow logins over the first serial port. Find the entry for `tty00`, which should look like this:

```
tty00   "/usr/libexec/getty std.9600"   unknown off
```

Remove the last two words, and replace them to match the following:

```
tty00   "/usr/libexec/getty std.9600"   vt220    on secure
```

Now run `kill -1 1`, and you should get a login prompt over your serial line.

Multiuser Startup

When the kernel finishes its core setup and hands control over to userland, `init(8)` runs the shell script */etc/rc*. This script handles all system setup, including mounting filesystems, configuring device nodes, identifying shared libraries, and any other task required to make the system usable. Some tasks are delegated to separate scripts; for example, */etc/netstart* is used to configure the network.

In this section, we'll cover how */etc/rc* and other startup scripts function, and the flow of the startup process. Armed with this understanding, you should be able to easily configure your OpenBSD machine to start exactly what you need—no more, no less.

Startup System Scripts

The startup system includes the scripts */etc/rc*, */etc/rc.conf*, */etc/rc.conf.local*, */etc/netstart*, */etc/rc.securelevel*, */etc/rc.local*, */etc/rc.shutdown*, */etc/rc.firsttime*, */etc/fastboot*, and the contents of the */etc/rc.d* directory.

The /etc/rc Script

On OpenBSD, everything outside the kernel is configured with a shell command, from setting the hostname to starting server daemons. The master script is */etc/rc*, and it runs all of these commands in the correct order, ensuring that the system is configured exactly the same way at every boot. As a final step, */etc/rc* runs `getty(8)` to present login prompts on all the appropriate terminals.

Never edit */etc/rc* unless you're a very experienced systems administrator with truly unique needs. This is one of the several files in */etc* that is technically editable, but mere mortals are well advised to *treat as binary*. Instead, whenever you need to disable functions, deactivate them in */etc/rc.conf.local*. To add new functionality to the startup process, use the shell scripts */etc/rc.securelevel* and */etc/rc.local,* or write a shell script for */etc/rc.d*.

The /etc/rc.conf Script

The */etc/rc.conf* file contains nothing but the default values for all other startup scripts. Read this file to see the configuration options for different system services. Here's a small snippet of what you'll find in *rc.conf*:

```
...
bgpd_flags=NO           # for normal use: ""
rarpd_flags=NO          # for normal use: "-a"
bootparamd_flags=NO     # for normal use: ""
rbootd_flags=NO         # for normal use: ""
sshd_flags=""           # for normal use: ""
named_flags=NO          # for normal use: ""
...
```

If a variable is set to `NO`, the associated service is disabled by default.

As you can see, OpenBSD turns off almost everything by default, with one exception: the SSH daemon. Setting the variable to a pair of quotes, as shown after each entry in the preceding snippet, is enough to enable most daemons, and most daemons will run just fine without any command-line flags. However, if a daemon requires a command-line argument in order to run, that argument will be shown as it is in the `-a` attached to `rarpd_flags`.

NOTE *At the risk of beating my dead server senseless, never edit* /etc/rc.conf *(treat as binary—remember?). It will be replaced wholesale during a system upgrade. Instead, place your local values in* /etc/rc.conf.local.

The /etc/rc.conf.local Script

I've mentioned this before, but I'm going to beat you over the head with it: Place your changes to *rc.conf* in *rc.conf.local*. Entries in *rc.conf.local* override the defaults in *rc.conf*.

For example, say that on a particular machine, you want to run `sshd(8)` with extra debugging, and you also want to run `named(8)`. Additionally, you want to run the time server `ntpd(8)`, and have it correct the time at boot by using the `-s` flag. After consulting the documentation for those programs, you add the following lines to *rc.conf.local*:

```
sshd_flags="-D"
ntpd_flags="-s"
named_flags=""
```

OpenBSD will start the programs with the flags specified here. If you specify invalid, incorrect, or incompatible flags, the daemon will print error messages to the console.

The /etc/netstart Script

While its name differs from the other scripts, */etc/netstart* is definitely a system startup script. It reads */etc/mygate*, */etc/myname*, and all the */etc/hostname.if* files, and uses the information in them to configure all network interfaces, bridges, routing, and so forth. The file */etc/rc* runs this script before starting any server daemons, network filesystems, and so on. In single-user mode, you'll run this script by hand to bring up the network.

The /etc/rc.securelevel Script

The */etc/rc.securelevel* shell script runs early in the boot process, before */etc/rc* raises the system securelevel, but after starting the network. Many programs, particularly those that touch the kernel or intimately affect the filesystem, will not run once the securelevel is raised. If you run such a program, you can add the command to start it to this script. If your local program doesn't need to run before the system securelevel is raised, you're better off starting it from *rc.local* or writing a proper *rc.d* script for it.

One important entry in *rc.securelevel* is the definition of the system securelevel. We'll discuss securelevels in Chapter 10. For now, don't touch the line that sets the securelevel unless you're already familiar with BSD-based systems and know exactly which toe you're shooting off.

The /etc/rc.local Script

After */etc/rc* does just about everything else, it runs */etc/rc.local*. You can put commands to start local daemons in *rc.local*, but you're better off writing an *rc.d* script to start local daemons so you can easily and consistently restart them later. Of course, if you're lazy, you can get by with *rc.local*.

The /etc/rc.shutdown Script

Whenever you use `reboot(8)` or `halt(8)`, OpenBSD runs the */etc/rc.shutdown* script, which you can count on to run extra commands needed to safely shut down your server. Most server software shuts down cleanly without any special intervention, but software that requires data integrity (like databases) may need help shutting down without losing data. Again, if at all possible, write an *rc.d* script to manage your software.

The /etc/rc.firsttime Script

/etc/rc runs the script */etc/rc.firsttime* once, mails the output to root, and deletes *rc.firsttime*. The installer uses *rc.firsttime* for tasks such as fetching firmware that can't be legally redistributed. While you won't normally use *rc.firsttime*, you should know that it exists and that you can use it to perform one-time tasks when a machine boots.

The /etc/fastboot Script

If the */etc/fastboot* file exists, OpenBSD assumes that all filesystems are clean (see Chapter 8), and the boot process skips checking filesystem integrity.

The /etc/rc.d Directory

The */etc/rc.d* directory contains shell scripts for managing software, as discussed in the next section. While the system comes with scripts for software included in OpenBSD, add-on packages can provide their own scripts (see Chapter 13).

Software Startup Scripts

OpenBSD uses shell scripts to start, stop, restart, check, and reconfigure server software. These scripts are found in the directory */etc/rc.d*. Every piece of server software that comes with OpenBSD has a script in this directory, as do most ports and packages that need scripts for proper startup and shutdown. Use these scripts to manage integrated software without rebooting the server.

The *rc.d* scripts read their configuration from *rc.conf* and *rc.conf.local*. Most servers run the SSH daemon `sshd`, which can be enabled by adding the line `sshd_enable=""` to *rc.conf.local*. Look in */etc/rc.d*, and you'll find the shell script *sshd*.

If you change your `sshd` configuration, you must restart the daemon. Use the shell script to do this consistently.

```
# cd /etc/rc.d/
# ./sshd restart
sshd(ok)
sshd(ok)
```

Of course, you could do the same thing without the shell script simply by identifying the currently running `sshd(8)` process, reading the man page

to see how to shut it down properly, and then restarting it with the same command-line flags. In the case of `sshd`, that's easy: Running `pkill -1 sshd` would tell the daemon to reread its configuration file. But restarting a daemon that requires all sorts of flags *is* a big deal. Automating these system administration tasks ensures that your daemons run consistently.

To see if a daemon is running, use the `check` command to check your shell for the return value. The script will return a `0` if the daemon is running and a `1` if it isn't, as shown here:

```
# ./nfsd check
# echo $?
1
```

As you can see by the `1`, `nfsd` is not running.

The most common use for `check` is in shell scripts. You can start the daemon with the argument `start` and terminate it with `stop`. Use the `restart` argument to tell the daemon to reload its configuration.

In OpenBSD, *rc.d* scripts run when the system boots and again when it shuts down. (Something needs to unmount all those hard drives, shut down daemons, and clean up.) At shutdown, every script in the */etc/rc.d* folder is called with the `stop` argument.

Third-Party rc.d Scripts

OpenBSD packages for third-party software include *rc.d* scripts as necessary. For example, the popular database server MySQL `mysql-server` package installs the script */etc/rc.d/mysqld*. To use the package, you must enable it in *rc.conf.local*:

```
mysqld_flags=""
```

Once the package is enabled, you can manage your MySQL server just like any other OpenBSD daemon. However, packaged software will still not start automatically at boot, so you must tell OpenBSD to run this particular *rc.d* script at boot and shut down with the `pkg_scripts` variable in *rc.conf.local*:

```
pkg_scripts="mysqld"
```

The startup process runs the scripts in this variable, in the order given, at boot. The order is important for certain daemons. For example, if you have a database-driven website, you should start the database before the web server. At shutdown, it runs these scripts in reverse order.

Force-Starting Software

Sometimes you don't want to enable software globally; you just want to run a certain daemon for a short time or to address a specific situation. You can use *rc.d* scripts to manage this software using the `-f` flag to force the software to run.

Now for a real-life example. I previously ran PostgreSQL on my server, but someone kidnapped my pet rats and blackmailed me into using MySQL in exchange for their safe return. I needed to check some data in the old database, however, so I force-started the disabled PostgreSQL server:

```
# ./postgresql -f start
postgresql(ok)
```

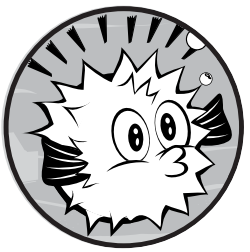
If you package or install your own software, I strongly recommend writing your own *rc.d* script. A few minutes spent reading the existing scripts will tell you most of what you need to know. For the rest, read the `rc.d(8)` and `rc.subr(8)` man pages.

Now that you can start OpenBSD, let's set up some user accounts.

6

USER MANAGEMENT

This one can log in,
this other can get email;
never give out root.



While computer intrusions over the Internet make headlines, a system administrator's greatest security threats often come from a system's own users. Maybe they won't ship your data to a crime syndicate, but disgruntled and incompetent users will crash your servers given the chance—sometimes out of malice, but more often out of ignorance. Think about security as the combination of confidentiality, integrity, and availability, and it will immediately become clear how users with unrestricted system access can damage security.

Despite what you might have learned from the Bastard Operator from Hell, the system exists for the users, and proper management of those users' accounts is absolutely necessary. In this chapter, we'll cover one of the systems administrator's most common tasks: managing users by adding, removing, configuring, and modifying user accounts.

The Root Account

In recent years, there has been a trend toward using the privileged root account for everyday tasks on systems that have only a single user.[1] Using a privileged account to read your email and browse the Web increases your risks from both user errors and malicious attacks. While a careless keystroke by a regular user will generate only a `permission denied` error, that same keystroke by root might render your system unusable and destroy all your data. Even if you're the only person using your OpenBSD system, you must use an unprivileged user account for day-to-day tasks.

If an intruder compromises an unprivileged account, the potential damage is limited only by that user's permissions. If the compromised account handles your email and web bookmarks, you might suffer only personal embarrassment. But if that account handles system administration tasks, your intruder can inflict unlimited system damage and send you scurrying for your backup. Using a regular account for day-to-day tasks means that you can take extra steps to restrict the root account.

Perform all tasks with the minimum level of privilege necessary. If you don't need root access to perform a task, don't use it! For example, OpenBSD's web server runs as the specific user `www`, rather than as root. If an intruder breaks into your web server and gains access your system as the `www` user, he can damage only the files the `www` user has permission to write to. Likewise, if the web server software goes into an error state and starts deleting files at random, this same principle limits the files it can delete. The least privilege approach protects the system from both intruders and its own software.

Operating systems that give every user privileged access have more problems as a result. Virus effectiveness, unexpected misconfiguration, and even crashes can be traced back to unnecessary privileged access. OpenBSD might be the most secure operating system in the world, but all those fancy security features can't protect you from poor system administration practices.

Using the root account for routine tasks also creates bad habits. People under pressure perform what they practice. If you use root on your desktop for routine work, you'll need to fight your habits to perform routine tasks when you work on a production server. This sort of sloppiness inevitably breeds security problems. Even on my OpenBSD desktop, where I'm the only user, I do everything as a regular user specifically to develop and maintain good sysadmin habits.

Adding Users

OpenBSD uses many of the standard UNIX user- and password-management programs, such as `passwd(1)` and `vipw(8)`. It also includes

1. This probably leaked through from the Microsoft culture, where for many years, every user had administrative access.

a friendly interactive user-creation program, adduser(8). We'll cover adduser first, and then look at some of the more advanced tools.

Adding Users Interactively

Only the root user can run adduser. If you start adduser at the command line without specifying any options, it drops you into a friendly interactive dialog where you can create new users.

Configuring adduser

The first time adduser runs, it asks a series of questions to determine its default settings. It saves these default settings, but you can change the defaults later.

```
# adduser
Couldn't find /etc/adduser.conf: creating a new adduser configuration file
Reading /etc/shells
Enter your default shell: csh ksh nologin sh tcsh [ksh]: ❶
Your default shell is: ksh -> /bin/ksh
Default login class: authpf bgpd daemon default staff
[default]: ❷
Enter your default HOME partition: [/home]: ❸
Copy dotfiles from: /etc/skel no [/etc/skel]: ❹
Send welcome message?: /path/file default no [no]: ❺
Do not send message(s)
Prompt for passwords by default (y/n) [y]: ❻
Default encryption method for passwords: auto blowfish des md5 old
[auto]: ❼
```

adduser first asks for your preferred default shell. It reads */etc/shells* to see all the shells installed on your system. Though I've long used tcsh, I usually start new users with the OpenBSD standard of ksh ❶. That way, they have a shell that more closely resembles what is used by the rest of the world, and they quickly learn that I cannot answer questions about their shell.

Next, adduser asks for your default login class. I'll cover login classes later in this chapter. For now, assign new users to the default login class at ❷.

If you have a default OpenBSD installation, your user home directories are on the */home* partition. If not, specify the default home directory at ❸.

User accounts need configuration dotfiles (*.shrc*, *.login*, *.profile*, and so on). If you have a directory containing customized dotfiles, tell adduser about it at ❹. Otherwise, just accept the default.

Though OpenBSD doesn't include a welcome message by default, you can put one on the system so new users will have an email waiting for them on their first login. Give adduser the full path to the file containing your welcome message at ❺.

Depending on how you create user accounts, you might want to provide a password when you create the user account. Accounts created without passwords are disabled until a password is assigned. If you won't be assigning passwords when creating accounts, you can tell adduser not to prompt you for them at ❻.

Finally, you can choose the encryption algorithm used to hash user passwords, which are stored in */etc/master.passwd*. Unless you have specific interoperability needs or otherwise know what you're doing, accept the default at ❼.

From now on, `adduser` will use these chosen defaults. If you want to modify the defaults later on, change them in */etc/adduser.conf*. Read the `adduser(8)` man page for a complete list of configuration file options.

Creating User Accounts

Now that you've set your default options, run `adduser` again to create user accounts.

Start by assigning a username. Many people are irrationally attached to particular usernames, and it's polite to ask them if they have a preference.

```
Ok, let's go.
Don't worry about mistakes. There will be a chance later to correct any input.
Enter username []: pkdick
```

Once you have a username, you'll get a chance to enter the user's real name or the account's intended purpose.

```
Enter full name []: Phil Dick
```

The shell you specify is a matter of user preference. The list of shells is taken from */etc/shells*, with the addition of the `nologin` option. Users can change their shell unless you specifically prevent that, so don't worry too much about which shell you assign.

```
Enter shell csh ksh nologin sh tcsh [ksh]:
```

Next, choose a user ID (UID) number. By default, UID numbering starts at 1000, and `adduser` uses the lowest available number. You can change this if needed to match some local standard.

```
Uid [1001]:
```

By default, new users are assigned to a group with the same name as their username. Each user can be assigned to only a single login group (or primary group), but you can assign user accounts to multiple secondary groups if needed. If you want this user to be able to use the root account, invite the user to the `wheel` group. Other common groups include `staff`, `users`, and `operator`.

```
Login group pkdick [pkdick]:
Login group is ``pkdick''. Invite pkdick into other groups: guest no
[no]: wheel
```

Choose a login class for the user. If you don't understand login classes yet, accept the default. I recommend assigning administrative users—for example, those in the `wheel` group—to the `staff` class. If you're a desktop user, you want to be in the `staff` login class.

```
Login class authpf bgpd daemon default staff [default]: staff
```

If you set `adduser` to ask for passwords, it will ask you for a password, and then ask again to confirm.

```
Enter password []:
Enter password again []:
```

Now `adduser` displays everything you selected.

```
Name:        pkdick
Password:    ****
Fullname:    Phil Dick
Uid:         1001
Gid:         1001 (pkdick)
Groups:      pkdick wheel
Login Class: staff
HOME:        /home/pkdick
Shell:       /bin/ksh
OK? (y/n) [y]: y
```

Either accept or reject the user at this point. If you accept, `adduser` will create the new user and ask if you want to create another user.

Adding Users Noninteractively

If you need to create many users, you probably don't want to spend your day looping through `adduser` dialogs. If you have scripts, cron jobs, or web interfaces that add user accounts, you'll want to create users noninteractively. `adduser`'s `-batch` flag enables this. When you use batch mode, `adduser` takes four additional arguments: the username, the groups the username belongs to, the full name, and the password in encrypted format.

```
# adduser -batch username group 'Real Name' encryptedpassword
```

To create our user `pkdick` in batch mode, we would run this:

```
# adduser -batch pkdick wheel 'Phil Dick' NotThePassword
```

One thing to note here is that `pkdick`'s password is *not* `NotThePassword`. `adduser` expects us to provide a random salt that hashes to the string `NotThePassword`, not the password itself. For instructions on how to generate encrypted passwords, see "Passwords and Batch Mode" on page 90.

Groups in Batch Mode

By default, new users are assigned a primary group with the same name as their login name. In batch mode, you must specify additional groups desired on the command line. Our example user `pkdick` is created with the login group of `pkdick`. If you want to set a different login group for a particular user, use the `-group` flag.

```
# adduser -group guest -batch jgballard customers 'Jim Ballard' NotThePassword
```

You'll need to add the user to another group. Here, I gave `jgballard` the login group of `guest` and added him to the group `customers`.

To assign a user to multiple groups, separate the groups using commas.

```
# adduser -batch jgballard customers,sftp-only 'Jim Ballard' NotThePassword
```

The end result here is that `jgballard` is assigned to the `jgballard` primary group and added to the `customers` and `sftp-only` secondary groups.

Passwords and Batch Mode

If you actually follow any of the previous examples, you'll create an account with no known password. Modern Unix-like operating systems don't store passwords in readable format; instead, passwords are stored as a hash of the password and a random salt. When you assign a password to a user, the system takes the password, adds the salt, and performs some horrible computations to generate a hash of the password. The system then stores that hash and salt in the */etc/master.passwd* file. When you attempt to log in, the login process takes your password, adds the salt, and computes the hash of that combination. If the computed hash matches what's stored in */etc/master.passwd*, the login is permitted.

The examples create an account with a password hash of `NotThePassword`. Because this isn't a legitimate hash, no entered password will match it. We need to provide a pregenerated encrypted password, enter an unencrypted password, and let `adduser` calculate the hash for us, or create an account without a password.

Creating a new account without a password is the simplest option. OpenBSD will disable the account until you assign a password to it, but this is acceptable for accounts used to run daemons, or if you have a help desk staff to assist new users in setting passwords. To create an account without a password, simply omit the password from the account-creation process.

```
# adduser -batch pkdick wheel 'Phil Dick'
```

If you want to enter an unencrypted password on the command line, use the `-unencrypted` option. Put this option before the `-batch` option. For example, to give Phil's account the password `IsThePassword`, enter the following:

```
# adduser -unencrypted -batch pkdick wheel 'Phil Dick' IsThePassword
```

This account now has a password of `IsThePassword`. You might use this inside a script or when no one is around to look over your shoulder. The password will appear in the system's process list, however, so any users on the system can see the password if they're quick enough to notice.

Another option is to generate a prehashed password using `encrypt(1)`. By default, `encrypt` gives you a blank line. When you enter a word, it returns the hash of that word. It defaults to using the encryption algorithm defined in the `default` login class. (For the past several years, this has been Blowfish.) You can enter any number of words, and each will be hashed separately. Press CTRL-C to exit `encrypt`.

```
# encrypt
gerbil
$2a$06$V/VO91VVAKSNslesQNH6pezXsGhoKUMcnvWxyDOJUmWRk3fflX5cy
weasel
$2a$06$652ngShUnOBuFEL7X2yrf.EOU2GUw/FseVq/BkVgaiyqvp4wt.Nsy
^C
#
```

If you're encrypting only one password or creating passwords interactively, give the `-p` option to `encrypt`. This gives you a non-echoing password prompt.

```
# encrypt -p
Enter string:
$2a$06$nyA.mygoei/6VGS2tq4wA.VOzB6inwlK9pWOIAsiUWBkWfOCqOJ7.
#
```

Other Batch Mode Options

I frequently create administrator accounts with one set of standards and unprivileged accounts with another. I create sysadmin accounts by hand using `adduser` in interactive mode (I don't create sysadmin accounts very often). Someone else creates unprivileged user accounts using an `adduser` batch mode script I wrote. *adduser.conf* contains the default settings for sysadmins, which I then override in the script. This approach requires less of my organic memory and ensures that unprivileged accounts are consistent.

All of these options must appear on the command line before the `-batch` argument. `adduser` treats everything after `-batch` as account information.

The `-noconfig` option tells `adduser` to not read defaults from *adduser.conf*. Using this option in a script guarantees that sysadmin-friendly defaults in *adduser.conf* don't leak into unprivileged accounts.

The `-dotdir` option specifies a directory for user dotfiles. All files in this directory are copied to the new user's home directory. I often have special dotfiles for unprivileged users.

The `-home` option tells `adduser` where to create the new user's home directory. This is not the actual home directory, but the base directory where the home directory will be created. For example, if all of your web server customers have home directories on the */www* partition, you might use `-home /www`.

To assign a nondefault login class, use the `-class` option.

The `-message` option gives a path to the new user message. To turn off a default of sending a message, use `-message no`.

To assign a shell, use `-shell` and the shell name as it appears in */etc/shells*, or `nologin` to disable logins.

Perhaps you want to assign your batch-created users UIDs in a specific range. Maybe all of your customers have a UID above 10000, while sysadmins have a UID in the thousands. Specify a minimum UID with `-uid_start` and a maximum with `-uid_end`. If available, the login group created will be given a GID equal to the UID.

User Account Restrictions

User accounts are subject to the following restrictions, fully documented in `adduser(8)`.

- Usernames can contain characters (preferably lowercase) and digits, as well as nonleading hyphens, periods, underscores, and a trailing `$`. Usernames can be no longer than 31 characters.
- Full names cannot contain a colon (`:`).
- Other values must exist in the relevant files: shells must appear in */etc/shells*, login classes in */etc/login.conf*, and so on.

Removing User Accounts

Removing unneeded user accounts is just as important as adding new ones. Use `rmuser(8)` to delete accounts.

```
# rmuser pkdick
Matching password entry:

pkdick:*:1001:1001::0:0:phil dick:/home/pkdick:/bin/ksh

Is this the entry you wish to remove? y
Remove user's home directory (/home/pkdick)? y
Updating password file, updating databases, done.
Updating group file: Removing group pkdick -- personal group is empty
 done.
Removing user's home directory (/home/pkdick): done.
```

The `rmuser` command displays the account entry from */etc/passwd*, giving you a chance to verify that you really want to delete this particular user. Read the account's real name, and verify that you're deleting the correct account. Next, `rmuser` asks if you want to delete the user's home directory. If you suspect that you might need some files from that user account, you could choose to keep the directory around for a while. It automatically deletes the user's cron jobs and incoming mail file.

Editing User Accounts

You create users with privileges based on the knowledge you have at the time. The information you have is probably wrong, so get comfortable with editing users. In most cases, `chpass(1)` does everything you need in a user-friendly way.

Users can edit their own accounts by running `chpass` without any arguments.

```
$ chpass
$ Changing user database information for mwlucas.
Shell: /usr/local/bin/tcsh
Full Name: mwlucas
Office Location:
Office Phone:
Home Phone:
```

Here, users can update their shell or change their directory information. Many applications ignore the directory information (phone numbers and office location) stored in */etc/passwd*, but in some places, it's important. Make changes, save, and exit.

If you run `chpass` as root, giving a username as an argument, you get a very different picture.

```
# chpass mwlucas
# Changing user database information for mwlucas.
Login: mwlucas
Encrypted password: $2a$08$s2EVX.cAhYHskOaHk/4C5eLn76atAmGPU7z5DqRKAYe/V.OGgWXVi
Uid [#]: 1000
Gid [# or name]: 1000
Change [month day year]:
Expire [month day year]:
Class: staff
Home directory: /home/mwlucas
Shell: /usr/local/bin/tcsh
Full Name: mwlucas
Office Location:
Office Phone:
Home Phone:
```

Here, you can forcibly change the user's password (although there are better ways to do this), shell, UID, password expiration, and so on, in addition to all of the user's directory information.

Changes made through `chpass` affect only */etc/passwd*, */etc/master.passwd*, and */etc/group*. If you change a user's UID, GID, or home directory, you must also make the corresponding changes to the files the user owns and his home directory; otherwise, the user's account won't work correctly. If */etc/passwd* lists your home directory as */newhome/mwlucas* in */etc/passwd*, but your files are in */home/mwlucas*, you'll have trouble on your hands.

Note that you can't edit */etc/master.passwd* or */etc/passwd* with just any text editor; you need to use tools that manage the corresponding password

databases. If you insist on editing the password file by hand, you can use `vipw(8)` to directly edit */etc/passwd*. If you're not familiar with `vipw`, stick with `chpass`. The most common use for `vipw` is when the password file is damaged, and the most common way someone damages the password file is by using `vipw`.

Login Classes

A user's shell can be used to limit what a user can do, but OpenBSD provides very specific access controls with login classes. Login classes, set in */etc/login.conf*, define the resources and information accessible to users. Login classes also let you control password length and expiration times, as well as external authentication mechanisms.

Each user is assigned to a class, and each class places limits on available resources. When you change the limits on a class, the new limits are applied to each user the next time the user logs in. Define a user's class when creating the account, or change it with `chpass`.

By default, *login.conf* offers two classes for users, one class for daemons, and a few special-case classes. The `default` user class gives the user wide-ranging access to system resources and is suitable for machines with a limited number of shell users. The `staff` user class gives the user no restrictions on memory use, sets very high limits on the number of processes a user can run concurrently, and allows the user to log in even when logins are forbidden.

If these two classes meet your needs, and if you won't be using an alternative authentication protocol like Remote Authentication Dial In User Service (RADIUS) or Kerberos, you can skip this section. If not, read on.

Login Class Definitions

Each class definition consists of a series of variable assignments describing the class's resource limits, authentication, and environment. Each variable assignment in the class definition begins and ends with a colon. The backslash character indicates that the class continues on the next line, which makes the file more readable.

Here's the definition of the `default` class:

```
default:\
❶      :path=/usr/bin /bin /usr/sbin /sbin /usr/X11R6/bin /usr/local/bin /usr/
local/sbin:\
❷      :umask=022:\
❸      :datasize-max=512M:\
       :datasize-cur=512M:\
       :maxproc-max=256:\
       :maxproc-cur=128:\
       :openfiles-cur=512:\
       :stacksize-cur=4M:\
       :localcipher=blowfish,6:\
       :ypcipher=old:\
❹      :tc=auth-defaults:\
       :tc=auth-ftp-defaults:
```

The `default` class has several variables. Some of these have fairly obvious interpretations. For example, the `path` variable at ❶ assigns a default command search path to the user's shell, usually visible to the user as `$PATH`. The `umask` setting at ❷ assigns a default umask to the user's shell. The user can override both of these.

Other settings, such as `datasize-max` and `maxproc-max` at ❸, are harder to define by guesswork. We'll go through some of the more commonly used values in the next section.

Similar in behavior to the `termcap tc` variables at ❹ used to configure serial console clients in Chapter 5, the `default` class copies settings from the entries `auth-defaults` and `auth-ftp-defaults` elsewhere in *login.conf*.

Some variables don't require a value to trigger behavior; these values trigger a specified behavior simply by adding them to *login.conf*. For example, the presence of `requirehome` means that the user must have a valid home directory to log in.

Changing login.conf

On many BSD systems, you must transform the *login.conf* file to a program-friendly database file, *login.conf.db*, with `cap_mkdb(8)`. OpenBSD doesn't require this. Programs that check login classes first look for the login class database, and if they don't find it, they directly parse *login.conf*. You can use `cap_mkdb` to create such a database, which will very slightly improve the performance of software that checks *login.conf*.

```
# cap_mkdb /etc/login.conf
```

Note that once you create this database, you must rebuild it every time you edit *login.conf*. Database values in *login.conf.db* will always override your *login.conf* settings. Alternatively, you can remove *login.conf.db* and force programs to always parse *login.conf*.

I recommend skipping `cap_mkdb` on modern hardware.

Legal Values for login.conf Variables

The *login.conf* variables accept only very specific values, including the following:

- A full path to a text file or a program
- A comma-separated list of environment variables
- A comma-separated list of values
- A number (put a 0x in front of the number for hexadecimal, or a 0 in front for octal)
- A space-separated list of pathnames
- A size, in bytes (default), kilobytes (K), megabytes (M), gigabytes (G), or 512-byte blocks (T)
- A time in some combination of seconds (assumed if no unit is given), minutes (m), hours (h), days (d), weeks (w), or years (y)

Variables that use pathnames accept the special symbols tilde (~) and dollar sign ($). A tilde followed by a slash or the user's login name, or at the end of a pathname represents the user's home directory. You can use ~/bin to represent a bin directory in the user's home directory. The dollar sign represents the username. For example, you might use /var/mail/$ to represent the user's incoming mail file.

Some variables require particular types of values. A path to the user's home directory must be a full path, while the amount of memory a user can allocate must be a size. In most cases, legitimate answers are fairly obvious, but check login.conf(5) for a full listing of acceptable values.

Setting Resource Limits

Resource limits allow you to control the amount of system resources any one user can monopolize at any one time. If several hundred users are logged in to one machine, and one user decides to compile LibreOffice, that person will consume far more than his fair share of processor time, memory, and I/O. By limiting the resources any one user can use, you can make the system more responsive for all users.

Resource limits were more commonly used back when computing facilities were very expensive and departments received bills for the amount of computing time they used. These days, utilization accounting isn't so important. It's generally cheaper to buy more computing power than it is to configure accounting or resource limits. That said, if you have a buggy daemon that sometimes leaks and starts to soak up CPU time or memory, giving it a login class can prevent it from devouring the system.

Table 6-1 lists some resource-limiting *login.conf* variables.

Table 6-1: Some login.conf Resource Limits

Variable	Description
coredumpsize	Maximum size of a core dump file
cputime	Maximum CPU time any one process can use
datasize	Maximum data size per process
filesize	Maximum size of any one file
maxproc	Maximum number of processes
memorylocked	Maximum locked-in core memory use per process
memoryuse	Maximum core memory use per process
openfiles	Maximum open file descriptors per process
stacksize	Maximum stack size per process
vmemoryuse	Maximum virtual memory use per process

Resource limits are generally set per process. If you permit each process 200MB of RAM and allow 40 processes per user, you've just allocated each user 8GB of memory. Perhaps your system has a lot of memory, but does it really have that much?

All resource-limiting variables except `vmemoryuse` support maximum and current (advisory) limits. Users are warned by the system when they exceed current limits and cannot exceed the maximum limits. This works well on a cooperative system, where multiple users share resources but need to be notified when they are approaching their limit.

To specify a current limit, add `-cur` to the variable name. To make a maximum limit, add `-max`. For example, to set a current and maximum limit on the number of processes a user can have, use this definition in the class:

```
...
:maxproc-cur: 50:\
:maxproc-max: 60:\
...
```

A user in this class will receive a warning when he uses more than 50 processes and will not be able to use more than 60 processes. If you do not specify a limit as current or maximum, it acts as both.

Modifying the Shell Environment

You can define environment settings in a user class. This can work better than setting them in the default shell profile, because changes affect all users immediately upon their next login. This setting will impact all user shells, even those that don't read *.profile* or *.cshrc*.

Table 6-2 lists popular user class variables that affect the user environment.

Table 6-2: Some login.conf Environment Variables

Variable	Description
`hushlogin`	If present, no system information is given out during login.
`ignorenologin`	User can log in even when */etc/nologin* file is present.
`nologin`	Path to a file. If the file exists, when a user tries to log in, the file contents are displayed and login is denied.
`path`	Default command search path.
`priority`	User's priority (nice) level. See `renice(1)`.
`requirehome`	If present, user must have valid home directory to log in.
`setenv`	A comma-separated list of environment variables and values.
`shell`	User shell. Overrides user shell selection in */etc/passwd*. The user's `$SHELL` environment variable reflects */etc/passwd*, resulting in an inconsistent environment. Playing games with this is an excellent way to annoy your users.
`term`	Default terminal type, if environment can't figure out terminal type.
`umask`	Initial umask. Should always start with a 0.
`welcome`	Path to a file containing the login message.

Password and Login Options

Unlike the user environment, which can be configured in several different places, many password controls can be configured only via the user class. The password controls affect only the local password database, not Lightweight Directory Access Protocol (LDAP), Kerberos, RADIUS, or other remote password databases.

Let's have a look at some commonly used password controls.

`localcipher`
: This controls the password hashing method used in */etc/master.passwd*. The default is Blowfish. Don't change the password hashing method unless you're trying to be compatible with a specific foreign Unix-like operating system. See `login.conf(5)` for the list of supported hashing algorithms.

`login-backoff`
: This controls how quickly a user can struggle to remember his password. After this many unsuccessful login attempts, `login(1)` slows down how quickly it offers a new username and password prompt.

`passwordcheck`
: This gives the full path to an external program that checks new passwords for quality. OpenBSD passes the password to the program on standard input. The program is expected to return a 0 if the password is adequate and a 1 if the password is inadequate. OpenBSD includes a very simple and limited password-quality checker; if you need a password-quality checker, check out `passwdqc` (*/usr/ports/security/passwdqc*).

`passwordtries`
: This is the number of times `passwd(1)` uses the password-quality checker. If the user cannot come up with a sufficiently complicated password in this many tries, the new password is accepted anyway. If this is set to 0, a new password is accepted only when it passes the quality check.

`minpasswordlen`
: This is the minimum length of a new password. Password length is not a measure of quality—a stream of 128 `A` characters is still a lousy password, but it might help you meet site requirements.

`passwordtime`
: This is the maximum age of a password, in seconds. Use this to require regular password changes.

`password-warn`
: This is the length of time, in seconds, before `login(1)` begins warning the user of an expiring password.

`password-dead`
: This is the length of time, in seconds, after password expiration when the user may log in one last time, just to reset his own password. If the user does not reset his password, he cannot log in. This is a last-chance grace period; if the user blows this chance, sysadmin intervention is required to reset the password.

Changing Authentication Methods

OpenBSD supports many different authentication mechanisms, such as the local password file, Kerberos, S/Key, RADIUS, and so on. Specify the authentication method desired in the user class definition, and OpenBSD will use it. This system behind this is called *BSD Authentication*.

Setting an authentication mechanism does not configure the authentication mechanism. For example, configuring a login class to authenticate via Kerberos doesn't magically establish a Kerberos domain. If the specified authentication method is unavailable, classes configured to use that method will be unable to log in.

Not all authentication methods interoperate with all protocols. For example, while SSH works with physical tokens, it doesn't work with the `lchpass` authentication protocol, which allows users to change their password but disallows logins. Review the man page for each authentication method for details.

Some authentication methods require additional configuration. For example, if you want to use RADIUS authentication, you must tell your system where to find your RADIUS server. The special *login.conf* variables and their use are documented in the authentication method's man page.

Table 6-3 lists the authentication methods supported by OpenBSD's built-in BSD Authentication.

Table 6-3: BSD Authentication Methods

Method	Man Page	Description
`activ`	`login_activ(8)`	Authenticate via ActivCard token
`chpass`	`login_chpass(8)`	Change password, no shell
`crypto`	`login_crypto(8)`	Authenticate via CRYPTOCard token
`krb5`	`login_krb5(8)`	Authenticate via Kerberos
`krb5-or-pwd`	`login_krb5-or-pwd(8)`	Try Kerberos, then local password database
`lchpass`	`login_lchpass(8)`	Change local password
`passwd`	`login_passwd(8)`	Authenticate against local password file
`radius`	`login_radius(8)`	Authenticate against RADIUS server
`reject`	`login_reject(8)`	Request a password, then deny the login
`skey`	`login_skey(8)`	Authenticate via S/Key
`snk`	`login_snk(8)`	Authenticate via SecureNet token
`token`	`login_token(8)`	Authenticate via X9.9 token
`yubikey`	`login_yubikey(8)`	Authenticate via Yubico YubiKey token

The ports collection (discussed in Chapter 13) contains a few additional login methods, such as fingerprint scanners (*sysutils/login_fingerprint*), OATH one-time passwords (*sysutils/login_oath*), and LDAP integration (*sysutils/login_ldap*). You can also create your own custom authentication methods; see `login.conf(5)` for details.

Set the authentication method using the `auth` variable in *login.conf*:

```
:auth=token,passwd:\
```

Users in a class with this set try to authenticate via an X9.9 token. If that's not possible, the system falls back on the local password database.

BSD Authentication supports different authentication methods for different daemons. You can specify a service name after the `auth` keyword, indicating that this set of authentication methods applies to only that particular service. You'll frequently see login classes like `auth-ssh` and `auth-su`.

Here are a couple of sample entries from the default *login.conf* file:

```
# Default allowed authentication styles
auth-defaults:auth=passwd,skey:

# Default allowed authentication styles for authentication type ftp
auth-ftp-defaults:auth-ftp=passwd:
```

This defines the class `auth-defaults`, with only one entry. By default, users in this class first use password authentication, and then S/Key authentication. The `auth-ftp-defaults` class defines `auth-ftp` as using the password database, and only the password database.

Earlier in this chapter, I mentioned that the default class included two other classes. These are the `auth-defaults` and `auth-ftp-defaults` classes. Every other login class in the default *login.conf* file includes them by reference. If you change the authentication methods used by the `auth-defaults` class, that change will apply to every other login class.

Using Login Classes for RADIUS Authentication

I have a long-running love/hate relationship with RADIUS. It's the lowest common denominator of authentication protocols. Just about every operating system and hardware device supports it, but it's a finicky protocol with innumerable edge cases. Luckily, configuring OpenBSD as a RADIUS client is simple. Any RADIUS server can provide authentication services for OpenBSD.

I encourage you to use another login service, such as LDAP or Kerberos, rather than RADIUS. But in certain cases, for certain users, RADIUS is adequate. RADIUS combined with Microsoft's Internet Authentication Service gives you easy password synchronization with the local Windows domain and reduces your support load.

First, read `login_radius(8)`, and then configure your RADIUS server to permit access from your OpenBSD host. To configure RADIUS authentication, you need the RADIUS server's IP address, the port RADIUS runs on, and a shared secret. (For historical reasons, it's best to specify the RADIUS port explicitly rather than relying on */etc/services*.) In our example, the RADIUS server is 192.0.2.2, the port is 1812, and the secret is the string `Insubordination88`.

First, create a directory to hold the server configuration file and set its permissions appropriately, as per `login_radius(8)`.

```
# mkdir /etc/raddb
# chgrp _radius /etc/raddb/
# chmod 755 /etc/raddb/
```

Now create the file */etc/raddb/servers*. This file should contain a server and its secret, each on one line. Our *servers* file has only one line:

```
192.0.2.2        Insubordination88
```

Now change *login.conf* to use RADIUS by default.

```
auth-defaults:\
        :auth=radius:\
        :radius-port=1812:\
        :radius-server=192.0.2.2:
```

The `auth-defaults` class is OpenBSD's default authentication class. If we change it, we change how every other class authenticates. We set the `auth` type to `radius`, and set the port and the server.

Immediately upon saving the file, OpenBSD will try to authenticate all user accounts against the RADIUS server. You might want to change the `auth-ftp` class to match.[2]

Until you confirm everything is working, keep an SSH session logged in as root so you can change *login.conf*. Otherwise, you might lock yourself out of the system, or at least out of the root account. If you can't get into the system, you'll need to reboot into single-user mode and edit *login.conf*.

Changing the authentication scheme for all users might not be desirable, either. You might want `authpf(8)` users to authenticate against RADIUS, but have users in the `staff` class authenticate against the local password database. Perhaps you don't want your root account to authenticate via RADIUS, so you need an `auth-su` login class that points at the local password database. Using login classes, you can configure user authentication to fit your specific needs.

2. Or you might not want to make this change. FTP transmits passwords in clear text, so you might want to use a separate password source for FTP connections. Why transmit passwords securely over one protocol, while transmitting them insecurely on a neighboring port?

Unprivileged User Accounts

An unprivileged user account is a user account with no privileges to any programs or files. Many programs run as unprivileged users or use unprivileged users to perform specific duties. These unprivileged users get only the rights needed to perform a limited task.

"Only the rights needed to perform a limited task" sounds like every user account, doesn't it? That's true, but the account used by the least privileged human being still has more rights than many programs need. Any user with shell access usually has a home directory. Users can create files in their home directory, run text editors, process email, run scripts, and compile (if not install) software. An average shell user needs these minimal privileges, but programs do not. By having a program run as a very restricted user, you control the amount of damage the software or intruders can do to the system.

OpenBSD includes several unprivileged users out of the box. Take a look at */etc/passwd*, and you'll see accounts like `sshd`, `named`, `_ntp`, and so on. These are all unprivileged accounts used by specific server daemons. Examine them, and you'll find several common characteristics.

Unprivileged users do not have normal home directories. Most share the home directory of */var/empty*, which is owned by root and contains nothing except a logging socket. Having a home directory the user cannot write to makes the account less flexible, but is good enough for most server daemons. If these users do own files on the system, file permissions are usually set so that the user cannot write to them.

Similarly, no one should ever log in to the system with these accounts. If the `named` user account is reserved for the DNS subsystem, why would anyone actually need to log in as that account? Unprivileged users are assigned a shell that specifically forbids logging in: */sbin/nologin*.

How does all this enhance system security? Let's pick on the web server, a common intrusion vector, as an example. OpenBSD runs its web server as the user `www`. Suppose an intruder discovers a security flaw in your website and can use this to make the web server execute arbitrary code. This is a security nightmare; our intruder can now make the server program do absolutely anything within its power. But what, exactly, is within the web server's power?

A command prompt permits much more mischief and mayhem than a website, so the intruder will probably try to access a command prompt on the system. The `www` user has a shell that specifically disallows a command prompt. While this doesn't categorically prevent the intruder from getting a command prompt, it does make it much more difficult.

But our intruder is clever. Through really excellent intrusion skills, he makes the web server open a high-numbered port that dumps clients into a root shell. He now has access to a command prompt and can wreak untold damage . . . or can he?

He has no home directory, and no permissions to create one. Any files he wants to store must go into a globally accessible directory such as */tmp* or */var/tmp*, increasing his visibility. The web server configuration file is not

owned by the www user. Even if the intruder has a path into the web server, he cannot reconfigure it. He can't change the website files, as the www user doesn't own them. The www user doesn't have access to anything on the system, actually. Additionally, OpenBSD's built-in web server chroots itself. Having broken into the web server program, the intruder now must escape the chroot and penetrate a privileged program.

Can he penetrate your system? Possibly, but it will be much more difficult. If he is specifically targeting you or your company, he might go to the trouble. If he is just looking for easy meat, however, he will probably give up and go bother someone running a Linux or Windows system.

Using unprivileged users doesn't solve all security problems, mind you. The compromised www user can view web application source files. If your application is badly written or has database passwords hardcoded into hidden files, you're still in trouble. But if you don't use poorly written applications and you've kept your system updated and patched, the intruder will have a very hard time penetrating the rest of your server.

The nobody Account

The first unprivileged account was nobody. It was created for use by the Network File System (NFS, discussed in Chapter 9) to map files owned by root on foreign systems. Decades ago, people started using nobody as a generic unprivileged user, running web servers, proxy servers, and other daemons as nobody. While this was better than running those programs as root, it's still poor practice. If an intruder penetrated one of those programs, he would gain access to all processes owned by nobody. Our hypothetical web server intruder would suddenly have access not only to the web server, but also to the database, NFS, or anything else running as nobody!

Every daemon that needs to run as a user needs its own unprivileged accounts—the whole point of using unprivileged users is to minimize the damage one piece of software can inflict. Use them liberally. OpenBSD provides discrete unprivileged users for services as small as finger(1) and the audio system. Follow this example.

_username

If you take a look at */etc/passwd*, you'll see that many unprivileged users have an underscore before their name, such as _syslogd, _ldapd, and _dhcp. This is an OpenBSD convention for identifying unprivileged users. Most add-on software also uses unprivileged usernames beginning with an underscore, such as _mysql and _postgresql.

Not all unprivileged usernames start with an underscore, however. Some of these are legacy users that OpenBSD retains for compatibility reasons, such as nobody. Others have a long history or support inflexible software, and changing them would be more annoyance than it's worth.

The presence of an underscore means that a user is unprivileged. The absence of an underscore means nothing; the user might be a normal

account or it might be unprivileged. If you create your own unprivileged users, you don't need to include a leading underscore, but doing so will help other system administrators understand what the user does.

Creating Unprivileged Users

Here are common settings used for unprivileged users. You can change any of these as needed for your application.

username Assign a username related to the user's functions, so that you'll easily recognize it. Giving an unprivileged user a username like `_fgcrl` might seem like a good way to conceal its purpose, but it will confuse your sysadmins and an intruder will quickly figure it out.

home directory */var/empty* is a common setting for unprivileged users.

shell */sbin/nologin* is a common setting for unprivileged users.

UID/GID Choose a specific range of UIDs and GIDs for your custom unprivileged users. OpenBSD reserves all UIDs below 1000 for system-assigned unprivileged users.

full name Assign a name describing the user's role.

password Use `chpass(1)` to assign the user a single asterisk as their encrypted password. This disables the account password.

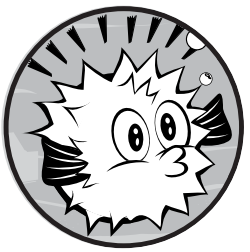
These settings make your unprivileged user very unprivileged indeed. You can set all of these options except the password using `adduser(8)`.

Now that you understand how to create, manage, and use user accounts, let's discuss how to manage privileged users.

7

ROOT, AND HOW TO AVOID IT

The root of all evil
is never far from your touch.
sudo saves your life.



The security of most Unix-like operating systems has long been considered coarsely grained. One superuser, root, can do anything. Other users are lowly sharecroppers who endure the shackles root places upon them. The problem is that root doesn't have many shackles and it can't individualize the ones that it has very well. Some operating systems use POSIX access control lists (ACLs) to provide more fine-grained access controls, but these are difficult to configure correctly.[1]

While it's true that Unix-like operating systems don't have detailed access controls, the fact is that most people don't bother using the controls that *do* exist. Fortunately, you can combine groups and permissions to handle almost any problem securely.

1. I could just say that "I have never seen POSIX ACLs configured correctly," but personal anecdotal evidence is not proof. Even the dozens of horrifying personal anecdotes I've gathered over decades in this business are not proof. Feel free to prove me wrong, but please, do it on *your* server.

The Root Password

The root user owns the system and has absolute power over every piece of hardware as well as certain actions that require absolute control, such as manipulating the kernel and changing authentication sources. You need root permissions to perform these tasks. You can log in as root, use `su(1)` to become root, or use `sudo(8)` (discussed later this chapter) to get certain root-level privileges without actually using the root account.

To use the root password, you can either log in as root at a console login prompt or, if you belong to the group `wheel`, log in as yourself and use the switch user command `su(1)`. Of the two, I recommend using `su`; it logs who uses it and can be used when you are logged in from a remote system. To use `su`, run the following:

```
$ su
Password:
#
```

When prompted, enter the root password. Now check your current user ID with `id(1)`.

```
# id
uid=0(root) gid=0(wheel) groups=0(wheel), 2(kmem), 3(sys), 4(tty),
5(operator), 20(staff), 31(guest)
```

As you can see, the UID is 0, which means that you now own the system, and I do mean *own* it. Consider every keystroke carefully. Carelessness could return your hard drive to its primordial, unformatted state.

Only users in the group `wheel` can use the root password to become root via `su`. If you give the root password to users without physical console access and who are in the `wheel` group, they can enter `su` and the root password as many times as they want, and it won't work. (But anyone can use the root account and password at the system console, so don't make a habit of blabbing the root password all around the office.)

Requiring group membership to use the root password leads to the question, "Who needs root access?" Root is required to configure many parts of OpenBSD, but once the system is running properly, you can greatly decrease or discontinue your use of root. For any remaining tasks that absolutely require root, use `sudo`.

Using Groups

One of the simplest ways to reduce the need for root is with groups. Unix-like operating systems classify users into groups, which consist of accounts of users who perform similar administrative functions. You can, for example, define a group named `dnsadmins` and add the accounts of every user who edits DNS zone files to that group. Then, by setting the permissions of the zone files and their directory appropriately, members of that group can

edit zone files and reload the name server without the root password. The good news is that you could create such a group for almost any system function, and thereby avoid giving those users root access. Using groups in this manner is a powerful and often neglected system administration tool. I use groups for administering my own servers—just because I *can* use root doesn't mean that I *want* to use root. Users can identify the groups they belong to by using `id(1)`.

```
# id
uid=1000(mwlucas) gid=1000(mwlucas) groups=1000(mwlucas), 0(wheel),
2005(dnsadmin)
```

My UID is 1000, and my username is `mwlucas`. My GID, the primary group ID, is also 1000 and is also named `mwlucas`. I'm also in the `wheel` and `dnsadmin` groups.

The /etc/group File

The file */etc/group* contains all group information. Each line contains four colon-delimited fields: the group name, password, ID number, and list of members.

```
wheel:*:0:root,mwlucas,pkdick
```

The *group name* is a human-friendly name for the group. This group is named `wheel`. Group names are completely arbitrary and you could call a group `lickspittles` if you want, but you should choose a name that gives an idea of the group's purpose. While you might remember that `lickspittles` can edit the company web page, will that group name make any sense to your coworkers? If it does, you probably need better coworkers.

The second field, the group password, was a great theory that became an appalling practice once exposed to the real world. Modern Unix-like systems don't do anything with the group password, but the field remains because old programs expect to find something here. The asterisk is just a placeholder to placate such software. (While OpenBSD could eliminate this field, some enterprises share */etc/group* across operating systems.)

The third field gives the group's unique numeric GID. Many programs use the GID rather than the name to identify a group. The `wheel` group has a GID of 0. The maximum GID is 232, or 4,294,967,296.

Last is a comma-separated list of all users in the group. As you can see, the users `root`, `mwlucas`, and `pkdick` are all members of the `wheel` group. To add users to a group, add their username to this list, but remember that no */etc/group* entry can contain more than 200 users or be longer than 1024 characters.

Creating Groups

In order to create a new group, you need a name and GID number. OpenBSD usually assigns the next free GID to a newly created group with GIDs below 1000 reserved for OpenBSD. Programs included in OpenBSD that

need a dedicated group ID use one below 1000. Software installed via the OpenBSD package system or ports (discussed in Chapter 13) assigns dedicated GIDs in the 500 to 1000 range. GIDs for user accounts start at 1000 and go up. If you create groups for special roles, start at a high GID so that these administrative groups will be obviously different from user accounts.

Groups, Unprivileged Users, and Group Permissions

The simplest way to create a new group is to use `adduser` to create an unprivileged user for the role, and use that user's group to assign file permissions. As with any other unprivileged user, give this account the home directory */var/empty* and a shell of `nologin`. Do not add this user to any other groups (especially not `wheel`). Lastly, let `adduser` disable the account. Sure, the shell will prevent logins, but an extra layer of defense won't hurt.

Now that you have an administrative user and a group, you can assign file ownership. A user and a group own every file. To view the permissions on existing files, including hidden ones, use `ls -la`. (If you've forgotten how file ownership and permissions work, read `ls(1)` and `chmod(1)`.) Many system administrators focus on file ownership and owner permissions, invest somewhat less time on worldwide permissions, and gloss over group permissions as if they don't exist. Look closely at the sample DNS files that follow.

```
# ls -la
total 22
drwxr-xr-x  2 mwlucas  wheel     512 Apr 16 22:02 .
drwxrwxrwt  8 root     wheel     512 Apr 16 22:00 ..
-rw-rw-r--  1 mwlucas  mwlucas 14595 Apr 16 22:02 michaelwlucas.com.db
-rw-r-----  1 mwlucas  wheel     198 Apr 16 22:02 rndc.key
```

This directory contains two files. The file *rndc.key* can be read and written by the user `mwlucas`; anyone in the `wheel` group can read *rndc.key*; and no one else can even read it. The file *michaelwlucas.com.db* can be read or written by the user `mwlucas` or anyone in the group `wheel`, but others can only read it. If you're in the group `mwlucas`, you can edit this file.

Say I want my junior DNS administrators to be able to edit zone files but not be able to edit the `rndc(8)` configuration. The file permissions are correct, but I need the files to be owned by my DNS administrative user, `dnsadmin`. I also want my DNS admins to be able to create new zone files, so they need write permissions on the directory. Here's how I would do that:

```
# chown dnsadmin:dnsadmin michaelwlucas.com.db
# chgrp dnsadmin rndc.key
# chown dnsadmin:dnsadmin .
# chmod 775 .
# ls -la
total 22
drwxrwxr-x  2 dnsadmin  dnsadmin    512 Apr 16 22:02 .
drwxrwxrwt  8 root      wheel      512 Apr 16 22:08 ..
-rw-rw-r--  1 dnsadmin  dnsadmin  14595 Apr 16 22:02 michaelwlucas.com
-rw-r--r--  1 root      dnsadmin    198 Apr 16 22:02 rndc.key
```

As you can see, these files are now owned by the user dnsadmin and the group dnsadmin. Anyone in the group dnsadmin should be able to edit *michaelwlucas.com.db* without using the root password. The user named—the unprivileged user for the DNS server—should be able to read both files. Add your DNS administrators to the dnsadmin group in */etc/group*, and they should no longer need the root password to do their jobs.

This model has limitations, however. While your junior admins can't accidentally edit *rndc.conf*, they can delete and replace it. It would be better to put that file in a directory they can read but not edit. And while our DNS administrators might think that they need the root password to restart the name server, they're wrong. Use rndc(8) to manage the DNS server; other tasks can be managed via cron(8) or through sudo.

Hiding Root with sudo

While the proper use of groups can almost eliminate the need for root access to edit files, that won't help with commands that can be run only by root. You could set up a cron job to reload the name server each day at midnight, but every piece of software occasionally needs a manual restart. Because root is an all-or-nothing affair, people who have one minor task to perform have traditionally needed the root password.

OpenBSD includes sudo(8) and its associated tools, which implement fine-grained access control for commands that can be run only by particular users. When configured properly, sudo lets normal users run specific programs as other users, including root. Configured improperly, sudo permits full root access. I'll explain a basic sudo setup that covers almost all uses, but remember that many more combinations are possible. And don't be afraid to read sudo(8), sudoers(5), and the documentation at the sudo home page (*http://www.gratisoft.us/sudo/*).

Why Use sudo?

The sudo tool provides benefits beyond fine-grained privilege control. Every command run via sudo is logged, making it very easy to track who did what. The senior sysadmin can change the root password and not give it out, even to people who have root-level access.

The sudo configuration file is designed to be shared across multiple systems, so one sudo policy can cover your entire network and every operating system. Admittedly, you'll have trouble using a single sudo configuration on operating systems with wildly unique directory layouts, such as Mac OS X, but you can easily share one configuration among OpenBSD, other BSDs, Linux, and even OpenSolaris or AIX.

sudo Disadvantages

The most common problem with sudo is getting your users to accept it. People who have historically had access to the root account think they "lose something" by working through sudo.

The key to overcoming this is to give users access only to what's required to perform the tasks for which they're responsible. A junior administrator who complains about insufficient privileges has either overreached his responsibilities or needs more privileges. One sure way to discover what people actually do is to implement a minimal `sudo` permissions scheme and wait for complaints. If no one complains, they're not working very hard.

The configuration syntax for `sudo` can be confusing because its configuration doesn't closely resemble any other configuration file, and getting everything right can be difficult at first. The configuration file is actually well suited to its purpose, however. Once you understand it, adjusting privileges is quick and easy.

NOTE *More seriously, a faulty `sudo` setup can create the appearance of security while leaving gaps for a user to become root. Be sure to test `sudo` every time you make a change, and avoid the common configuration mistakes I document here.*

Some users will do their best to push the limits of their access, for no other reason than to see if they can outsmart you. These users are best managed with a combination of careful configuration, administrative policy, and a cricket bat.

An Overview of the sudo Software

The `sudo` program is a `setuid` root wrapper that can run commands as any other user. Use `sudo` by giving it the command you want to run.

```
$ sudo /etc/rc.d/named restart
```

The `sudo` software compares the desired command (in this case, `/etc/rc.d/named restart`) to its internal list of permissions and privileges. If the configuration file allows that particular user to run that command as root, `sudo` runs it as root. And, because root can run any command as any user, `sudo` can also run commands as any arbitrary system user. You can use this fact to grant any user the ability to run specific commands as chosen users; for example, administrators of certain database servers must frequently run commands as the database user.

The `sudo` software is a suite with four pieces. The first piece is the actual `sudo(8)` command, the `setuid` root wrapper. The second is the configuration file */etc/sudoers*, which describes who may run which commands as what user. (*/etc/sudoers* is fully documented in `sudoers(5)`.) Third is the `visudo(8)` command that opens */etc/sudoers* in an editor and checks the configuration file syntax before exiting. Finally, the `sudoedit(8)` program is specifically for editing files as another user.

The visudo(8) Command

If */etc/sudoers* contains incorrect syntax, `sudo` will not run. If you rely on `sudo` to provide root-level access to the system and you break your *sudoers* file,

you'll lock yourself out of the root account and lose the ability to correct your error. That is bad.

Fortunately, the `visudo(8)` program provides some protection against this sort of error by locking */etc/sudoers* so that only one person can edit the configuration at a time. It then opens */etc/sudoers* in a text editor (vi by default, but it respects the `$EDITOR` environment variable). Make your changes and save your work. When you exit the editor, `visudo` should parse the file for syntactic correctness.

If `visudo` detects an error, it prints out the offending line number and asks what you want to do.

```
>>> /etc/sudoers: syntax error near line 34 <<<
What now?
```

Here, I've made an error near line 34. I can reedit the file to fix the error, quit without saving any changes, or force `visudo` to accept this file.

Press the E key, and `visudo` should return you to the editor. Go to the offending line, fix your error, save the file, and exit the editor again.

Enter the X key, and `visudo` should quit and revert the configuration file to its original state. Your changes will be lost, but that might be acceptable. It's better to have an old, working configuration than a new, broken one.

Pressing Q forces `visudo` to accept the file, busted syntax and all. If `sudo` can't parse */etc/sudoers*, it will immediately exit. Essentially, you're telling `visudo` to break `sudo` until you log in as root to fix the problem. If you think you understand */etc/sudoers* better than `visudo` does, you're probably wrong. Even if you're right, you're wrong.

The `visudo` program doesn't guarantee that the configuration will do what you desire, only that the configuration parses and is valid. A properly formatted configuration that declares "No one may do anything via `sudo`" is perfectly acceptable to `visudo`.

The /etc/sudoers File

The */etc/sudoers* file determines who may run which commands as which users. Never edit */etc/sudoers* directly, even if you think you know exactly what change you want to make. Always use `visudo` to change */etc/sudoers*.

The various *sudoers* sample configurations you'll find are usually very complicated, as they demonstrate all the nifty things `sudo` can do. At this point, however, you want to do only simple, boring things, like giving particular users access to run specific commands. And the bare syntax is very simple. Every *sudoers* rule follows this format:

```
Username    host=command
```

- The *username* is the username of the user who may execute the command, an alias for usernames, or a system group.
- The *host* is the hostname of the system this rule applies to. You can share */etc/sudoers* across multiple systems. This entry permits per-host rules.

- The *command* space lists the commands this rule applies to. You must list the full path to each command, or `sudo` will not recognize it. If this weren't a requirement, some untrustworthy soul could just adjust his `$PATH` to access renamed versions of commands.

For example, suppose I trust user `sbaxter` to run any command, on any system, as root. I use the keyword `ALL` to match all possible options for host and command:

```
sbaxter    ALL=ALL
```

As the lead sysadmin, I should know which duties I have assigned `sbaxter`, and exactly which commands he needs. Suppose `sbaxter` is my DNS minion. I control the actual editing of zone files with group permissions, but there are many legitimate occasions for him to stop, restart, or otherwise slap around the name server program. I want him to use the system script */etc/rc.d/named* for this task, and this *sudoers* entry gives him permission to use the script on all machines.

```
sbaxter    ALL=/etc/rc.d/named
```

If I share this file across several machines, it's likely that many of those machines don't even run a name server. To restrict my minion's access to only the DNS server, I'll change the host field.

```
sbaxter    dns1=/etc/rc.d/named
```

Then again, `sbaxter` is the administrator of the email server `mail1`. This server is his responsibility, so he needs to run any command. I can set entirely different privileges for him on the mail server and still use the same *sudoers* file on all the systems.

```
sbaxter    dns1=/etc/rc.d/named
sbaxter    mail1=ALL
```

Yes, `sbaxter` can use `visudo` on `mail1`, but he already has full privileges on that machine. I'm comfortable with this, as he knows I'll hold him responsible for any downtime.

Multiple Entries in a sudoers Field

Separate multiple entries in a single field with commas. For example, after a while, I get tired of `sbaxter` asking me to mount NFS shares on the DNS server, so I add `mount_nfs` to his privileges.

```
sbaxter    dns1=/etc/rc.d/named,/sbin/mount_nfs
```

He can now mount his own blasted NFS shares and leave me alone.

Running Commands As Non-root Users

Specify a username in parentheses before a command to say that the user can use `sudo` to run commands as a particular user. For example, my user `dwsmith` is a database administrator and needs to run any command as the user `_postgresql` on the database server `db1`.

```
dwsmith    db1 = (_postgresql) ALL
```

The `_postgresql` user can't successfully run critical system programs like `fdisk` and `newfs`, but it can restart the database, back it up, and perform other database-administration tasks. By choosing a specific user, a specific machine, and a specific command, you can define arbitrarily complex *sudoers* policies.

Long Lines

If you have several commands, usernames, or hosts on a line, that line might become uncomfortably long. Use a backslash (`\`) to indicate that a rule continues on the next line.

```
sbaxter    dns1=/etc/rc.d/named,/sbin/mount_nfs, \
       /sbin/reboot, /sbin/dump
```

Use as many lines as you like to make your *sudoers* file easier to manage.

/etc/sudoers Aliases

Take several machines with different roles, add multiple sysadmins with differing privilege levels, and your */etc/sudoers* file will quickly become complicated. When you have a few users with identical privileges and long lists of commands that you would like them to access, maintaining consistency in each user's privilege list becomes tedious. *Aliases* simplify these tasks and make */etc/sudoers* much more comprehensible, which makes your life easier.

An alias is a group of users, hosts, or commands. You can use aliases anywhere you would normally use users, hosts, or commands. You might, for example, create an alias called `DATABASE_COMMANDS` that contains all of the commands your database administrators need to run using `sudo`.

Let's take database administrator `dwsmith` and use an alias to specify his commands.

```
dwsmith    db1 = (_postgresql) DATABASE_COMMANDS
```

This alias might not seem to save us much, but suppose we have several database administrators. We could create an alias called `DBAs` that includes all of them.

```
DBAs    db1 = (_postgresql) DATABASE_COMMANDS
```

Suddenly, this one line represents multiple rules. All of the database admins have identical `sudo` privileges, and when you discover that you need

to give them access to an additional command, add the command to the alias, and it immediately becomes available to every database admin. There's no tedious and error-prone copying of entries between users.

You must define an alias before you can use it, so aliases normally appear at the top of the file. Each alias is made up of a label identifying its type, a name, and a list of its items. Alias types include user aliases, run as aliases, host aliases, and command aliases.

User Aliases

A *user alias* is a group of users, and it is labeled with the string `User_Alias`. Put only usernames in this alias.

```
User_Alias    DBAs = dwsmith, kkrusch
```

Here, the user alias `DBAs` contains the users `dwsmith` and `kkrusch`. By using the alias in my *sudoers* rules instead of the usernames, I ensure that these users receive exactly the same `sudo` privileges.

You can use system groups in user aliases by prefacing them with a percent sign (`%`). I might create a group in */etc/groups* called `databaseteam`, and make `dwsmith` and `kkrusch` part of that team.

```
%databaseteam db1 = (_postgresql) DATABASE_COMMANDS
```

Perhaps the most common usage of this is giving the `wheel` group unlimited `sudo` access.

```
%wheel ALL = ALL
```

This rule permits the `wheel` group to run any command as root through `sudo`. It doesn't change the group members' privileges, but gives them access via `sudo`. This is convenient for running single commands.

Run as Aliases

A *run as* alias is a list of users that other users can run commands as. For example, on certain application servers, the database admins need to run commands both as the database owner `_postgresql` and as the web server owner `www`. If the user must run commands as multiple users, however, you would need a separate *sudoers* entry for each target user.

A run as alias lets you group these accounts:

```
Runas_Alias    APPOWNER = _postgresql, www
```

You can now write a single rule allowing users to run commands as either `_postgresql` or `www`.

Host Aliases

A *host alias* is a list of hosts, defined as hostnames, IP addresses, or network blocks. Label host aliases with the string `Host_Alias`. Here are examples of all host alias types:

```
Host_Alias    DB = db1, db2, db3
Host_Alias    DMZ = 192.0.2.0/24
Host_Alias    FIREWALL = 192.0.2.1, 192.0.2.2, 192.0.2.3
```

NOTE *I warn elsewhere in this book about how security rules based on a hostname are vulnerable to DNS spoofing attacks. An intruder can't spoof the machine's local hostname, however, so you can safely use the hostname from* /etc/myname *in* sudoers.

Command Aliases

A *command alias* is a list of commands. For example, you might have a command alias that includes all of the commands needed to back up the system or restore from a backup. They're labeled with the string `Cmnd_Alias`.

```
Cmnd_Alias    BACKUPS = /bin/mt, /sbin/restore, /sbin/dump
```

You can tell a command alias to include everything in a particular directory by using a wildcard.

```
Cmnd_Alias       APPCOMMANDS = /home/appuser/bin/*
```

You can also list partial command names. For example, most of PostgreSQL's commands begin with the `pg_` prefix. To give a user access to these commands, use a wildcard like so:

```
Cmnd_Alias    APPCOMMANDS = /home/appuser/bin/*,/usr/local/bin/pg_*
```

If you find yourself writing command aliases that include paths like */sbin/**, stop and reconsider, because you're essentially giving the user unlimited root access.

Using Aliases in /etc/sudoers

Use an alias exactly as you would normally list the user, command, or hostname. In the previous examples, I defined the user alias `DBAs`, the run as alias `APPOWNER`, the host alias `DB`, and the command alias `APPCOMMANDS`. Here's how they might be used:

```
DBAs    DB = ALL
```

Here, the user group DBAs can run any command on any server in the DB group, as any user. The members of the group own the servers, and if they screw them up, it's not my problem.

Well, this attitude sounds good, but the truth is that when they destroy the server, I must get involved. Even if it's not my fault that they drove the database server into the ditch, it *is* my problem. I must lock down the commands that they can run, restricting them to only the commands in the APPCOMMANDS alias. So, the DBAs group can now run any command in APPCOMMANDS on the DB servers.

```
DBAs    DB = APPCOMMANDS
```

Then I discover that my database admins are either cleverer or dafter than I thought. They run certain database commands as root, creating log files owned by root. The unprivileged database user _postgresql cannot write to these log files, and so the application server crashes. Fixing this requires changing the permissions on those log files, but the database admins do not have permission to run chown. If I give them the ability to change the permissions on arbitrary files, I might as well just give them root access.

To keep this from happening again, I restrict their privileges so they can run their commands only as the application unprivileged users.

```
DBAs    DB = (APPOWNER) APPCOMMANDS
```

Everyone in the DBAs group can run any command in APPCOMMANDS, as any user in APPOWNER, on any server in DB. I can change their access by adding entries to the various aliases.

Without aliases, what would this rule look like?

```
dwsmith,kkrusch    db1,db2,db3 = (_postgresql,www) \
     /home/appowner/bin/*,/usr/local/bin/pg_*
```

That's ugly, and it does exactly the same thing.

If you name your aliases well, you'll find rules easier to understand. While these example aliases are fairly short, I've used aliases with up to 20 members. The resulting rules are appalling without aliases.

NOTE *Some of the permissions granted by sudo in this case are unnecessary. For example, the unprivileged web server user doesn't need to run the various PostgreSQL utilities, and if www did try to run the database, nothing much would happen. If you don't like this, make two separate rules. Either way, it's tighter security than giving database administrators the root password.*

Nesting Aliases

You can include aliases in aliases. Here, I combine two user aliases into a single alias for my application administrators:

```
User_Alias    APPADMINS = DBAs, WEBMASTERS
```

Alias Naming Conventions

It's traditional, but not mandatory, to give aliases names in all capital letters to help differentiate them from users, hosts, and so on. And though it's valid syntax, it's best to avoid naming aliases after users or hosts. Here's an example:

```
User_Alias    MWLUCAS    = mwlucas,pkdick,sbaxter,dwsmith
```

This would quickly drive me batty.[2]

You can also reuse alias names if they are for different types of aliases. For example, the following is perfectly legal, but perfectly offensive.

```
User_Alias    DB = dwsmith,kkrusch
Runas_Alias    DB = _postgresql,www
Host_Alias    DB = db1, db2, db3
Cmnd_Alias    DB = /usr/local/bin/pg_*, /home/appowner/bin/*
DB    DB = (DB) DB
```

If you do this, anyone who must debug your `sudo` configuration will curse your name. Even if you consider being cursed a job perk, this naming scheme makes your phone ring at inconvenient times.

Changing sudo's Default Behavior

You can customize `sudo`'s behavior, or its behavior for certain users, hosts, or aliases, with the `Defaults` field. For example, one feature of `sudo` is that if you enter the wrong password, it insults you.

```
$ sudo -l
Password:
My pet rat can type better than you!
Password:
```

I typed my password incorrectly. `sudo` insulted me and offered me a chance to enter my password again. If I enter the wrong password three times, `sudo` exits.

Insulting the user is just fine in an open source environment, but if you're in a company, someone will complain to management. You can either go to sensitivity training or proactively disable insults by adding the following line to *sudoers*:

```
Defaults !insults
```

The `Defaults` statement indicates that the following item affects one or more `sudo` defaults. The `insults` option controls insulting the user. The exclamation point (`!`) is a negation symbol. By putting an exclamation

2. Oh, all right—battier. Happy?

point in front of the option, you turn off the feature. The system will no longer insult users when they demonstrate that they cannot type as well as my pet rat.

```
$ sudo -l
Password:
Sorry, try again.
Password:
```

You can override defaults globally or on a per-alias basis.

Overriding Defaults per Host

To override the defaults on a per-host basis, use an `@` symbol after `Defaults` and give either a host or a host alias. Here, I want to insult users who can't type their password on `caddis` or on a machine in the alias `APPSERVERS`, while leaving insults disabled for all other servers:

```
Defaults !insults
Defaults@caddis insults
Defaults@APPSERVERS insults
```

This lets me enable or disable functions for any combination of servers.

Overriding Defaults per User

To change `sudo` defaults on a per-user basis, use a `%` and the user or user alias.

```
Defaults !insults
Defaults%lasnyder insults
Defaults%DBAs insults
```

It doesn't matter where `lasnyder` logs in—I'm going to insult him, as well as the users in the `DBAs` alias. But database administrators are used to poor treatment by their software, and to not insult them would confuse and disappoint them.

Overriding Defaults per Command

You can also change how `sudo` behaves on a command-by-command basis by putting an exclamation point between `Defaults` and the command list.

```
Defaults !insults
Defaults!/sbin/newfs,/sbin/fsck insults
Defaults%APPCOMMANDS insults
```

Anyone who tries to use `newfs(8)` or `fsck(8)` (discussed in Chapter 8) and cannot type their password needs insulting. The application administration commands might not merit insults, but I can always claim it was an oversight.

Overriding Defaults per Run As

Lastly, you can change the defaults based on who the command is being run as. Use a right angle bracket (<) to indicate changing behavior for a run as alias.

```
Defaults !insults
Defaults<_postgresql insults
Defaults<APPOWNER insults
```

If a user runs a command as `_postgresql`, or as any user in the `APPOWNER` run as alias, and types his password incorrectly, he gets insulted.

NOTE *In the rest of this chapter, we'll use `Default` widely. Please assume that each section includes the text "Restrict this as necessary by user, host, command, or run as."*

sudo and the Environment

Certain environment variables can cause problems. For example, `$HOME` is an obvious one—a user cannot create files in another user's home directory. Others, such as `LD_LIBRARY_PATH`, can cause endless annoyance as well as security issues, as applications try to link against the wrong libraries. The `sudo` program can remove suspicious environment variables, completely reset the user's environment, or be configured to preserve the original user's environment.

The `env_reset` *sudoers* option is set by default. It purges all environment variables except `LOGNAME`, `SHELL`, `USER`, `USERNAME`, and anything beginning with `SUDO_`. You can change this behavior by disabling `env_reset`, but I strongly recommend against disabling environment purging.

Instead of letting users blindly carry all the random garbage in their environment along with them, create a list of necessary and safe environment variables that they can retain. You'll see examples in OpenBSD's default *sudoers* file using the `env_keep` option.

```
Defaults env_keep +="DESTDIR DISTDIR EDITOR FETCH_CMD FLAVOR FTPMODE GROUP MAKE"
Defaults env_keep +="MAKECONF MULTI_PACKAGES NOMAN OKAY_FILES OWNER PKG_CACHE"
Defaults env_keep +="PKG_DBDIR PKG_DESTDIR PKG_PATH PKG_TMPDIR PORTSDIR"
Defaults env_keep +="RELEASEDIR SHARED_ONLY SSH_AUTH_SOCK SUBPACKAGE VISUAL"
Defaults env_keep +="WRKOBJDIR"
```

The OpenBSD team deems these environment variables safe to pass into a new user account. The `+=` means "add these to the existing list of items to keep." The environment variables themselves are in quotation marks.

If you need to pass your SSH environment around your servers, you can use `scp(1)` and `sftp(1)` to move files to other servers. Read the documentation, create a list of approved environment variables, and add an entry.

```
Defaults env_keep += "SSH_CLIENT SSH_CONNECTION SSH_TTY SSH_AUTH_SOCK"
```

NOTE *The ability to copy files to other servers probably should be restricted to people in a certain group. Sysadmins might need to copy files to other servers, but many other users don't need this access.*

Using sudo

Now that you know how to set `sudo` permissions, let's see how to actually use it. First, let's tell `sudo` that your account has permission to run any command. (You should have root access on your test machine, at least, so this won't be a security issue.)

The easy way to accomplish this is to uncomment the *sudoers* entry allowing `wheel` members access to all commands.

```
%wheel ALL=(ALL) SETENV: ALL
```

As a user in `wheel`, check your `sudo` permissions.

```
$ sudo -l
Password:
Matching Defaults entries for mwlucas on this host:
    env_keep+="DESTDIR DISTDIR EDITOR FETCH_CMD FLAVOR FTPMODE GROUP MAKE",
    env_keep+="MAKECONF MULTI_PACKAGES NOMAN OKAY_FILES OWNER PKG_CACHE",
    env_keep+="PKG_DBDIR PKG_DESTDIR PKG_PATH PKG_TMPDIR PORTSDIR",
    env_keep+="RELEASEDIR SHARED_ONLY SSH_AUTH_SOCK SUBPACKAGE VISUAL",
    env_keep+=WRKOBJDIR

User mwlucas may run the following commands on this host:
    (ALL) SETENV: ALL
```

When `sudo` asks for a password, enter your own password, not the root password.

The `-l` flag tells `sudo` to show you which privileges and settings you have. In response, `sudo` parses */etc/sudoers* and spits out all of the settings that apply to your account on this system. Any host-specific limitations are already evaluated and do not appear.

sudo Password Caching

When you enter your password correctly, `sudo` records the time, and for the next five minutes, it remembers that you've recently entered your password and will work without requiring you to enter it again. After five minutes, you must reauthenticate. This simplifies work when entering a series of `sudo` commands, but it times out reasonably quickly.

You can tell `sudo` to forget your cached password by running `sudo -k`. You can control the number of minutes before `sudo` asks for the password again with the `timestamp_timeout` option in *sudoers*. Here, we tell `sudo` to not time out the password for 10 minutes:

```
Defaults timestamp_timeout 10
```

If you set the timeout to 0, `sudo` always asks for a password. If you set it to a negative value, `sudo` caches the password throughout this login session. You must run `sudo -k` to make `sudo` forget that you entered your password.

Running Commands Under sudo

To run commands via `sudo`, just put the command name after the `sudo` command. For example, here's how you would run `tcpdump` via `sudo`:

```
$ sudo tcpdump
```

The `sudo` command should prompt for your password. Enter it correctly, and `tcpdump` should run as root.

You can also run commands that include arguments under `sudo`. For example, I use `tail -f` to view the end of a log file and show new entries as they appear. But some log files are accessible only to root, such as the authentication log and the log that contains detailed `sudo` logs. You can view these logs without becoming root by using `sudo`.

```
$ sudo tail -f /var/log/authlog
```

You can configure *sudoers* to permit any combination of commands and arguments.

Running Commands as Other Users

Earlier, you saw how to give some users permission to run commands as users other than root. Specify the user with the `-u` flag.

```
$ sudo -u _postgresql pg_dump
```

If you don't have permission to run that command as that user, you'll get an error.

sudoedit

My flunky `sbaxter` needs to edit the *named* configuration file, */etc/named.conf*. Consider this `sudo` configuration:

```
sbaxter    dns1=/etc/rc.d/named,/sbin/mount_nfs,/usr/bin/vi /etc/named.conf
```

Looks good, right?

Uh, no.

The first problem is that I'm requiring `sbaxter` to use a specific editor. Minimal competence in vi is required for system administrators, but I don't want to force him to use a specific editor to do his day-to-day job. Also, many editors offer shell escapes. While most people are aware of escaping

to a shell in vi, emacs has a shell escape as well. If my flunky can escape to a shell while running an editor as root, he gains root access. This is exactly what I want to avoid.

The sudoedit feature lets users edit specific files with their preferred editor, or a default chosen by the sysadmin, without working as root.

```
sbaxter    dns1=/etc/rc.d/named,/sbin/mount_nfs, \
         sudoedit /etc/named.conf, /etc/rndc.key
```

The keyword sudoedit is followed by a list of the files that the user can edit, thereby permitting the user to change those files without root privileges.

The user edits the file by passing a filename to sudoedit.

```
$ sudoedit /etc/named.conf
```

Technically, the user doesn't edit the actual file; instead, sudoedit copies the file to a temporary file owned by the user, and when the user closes the editor, it copies the temporary file to the original location. The user never runs the editor as root.

The sudoedit keyword uses the editor given by the environment variable $SUDO_EDITOR, $VISUAL, or $EDITOR. Users can set that variable in their shell if they don't like what the system offers them.

The Biggest sudo Mistake: Exclusions

Now that you know the basics of sudo, let's consider a configuration that trips up even experienced system administrators. Sometimes you want to prevent users from executing specific commands but give them access to every other command. The *sudoers* documentation says that you can do this using the exclamation point (!) as a negation character, but that's not entirely effective. Because this is a popular method, however, I'll discuss how it works, and then demonstrate how your users automatically get root if you use it.

Start by defining command aliases that contain the forbidden commands. One popular exclusion is su. Another common exclusion is user shells, because if you execute a shell as a user, you become that user.

```
Cmnd_Alias    SHELLS = /bin/sh,/bin/csh,/usr/local/bin/tcsh
Cmnd_Alias    SU = /usr/bin/su
```

Now configure a command alias that excludes those commands.

```
pkdick  ALL = ALL, !SHELLS,!SU
```

Looks sensible, doesn't it? And it seems to work.

```
$ sudo sh
Password:
Sorry, user pkdick is not allowed to execute '/usr/bin/su' as root.
```

Here's the catch: Commands are defined by full paths. You're allowing the user to run any command except for a few specified by full path. All this user needs to do is copy the command to another location and run it.

```
$ cp /bin/sh /tmp/sh
$ sudo /tmp/sh
#
```

Welcome to root!

Negating commands can be bypassed by anyone who understands even the basics of `sudo`, as you'll find well documented in the `sudo` manual and other literature. People *still* insist on using it to protect production systems. Don't be one of those people.

sudo Logs

Every `sudo` command is logged to */var/log/secure* by `syslogd`. Each log message contains a timestamp, a username, a terminal, the directory where the command was run, the user the command was run as, and the command used.

```
Apr 30 14:16:50 treble sudo:  mwlucas : TTY=ttyp8 ; PWD=/home/mwlucas ;
USER=root ; COMMAND=/usr/bin/su -m
```

By checking the file *secure*, you can track exactly who did what and when. (Send your *syslog* messages to a logging server that your users cannot access to prevent those who screw up from deleting the logs of their screwup.)

```
May 15 09:14:55 treble sudo:  lasnyder : TTY=ttyp4 ; PWD=/etc ; USER=root ;
COMMAND=/bin/rm pf.conf
```

I know exactly who broke this system and when. The log entry transforms what's about to happen from "homicide" to "justifiable manslaughter." That alone makes `sudo` worth using properly.

This chapter has given you some tips on how to avoid screwing up your system accidentally. Now let's look at ways to really mess up your system, by mucking with disks and filesystems.